

```
// ===== 文件: entryability/EntryAbility.ets =====
import { AbilityConstant, ConfigurationConstant, UIAbility, Want } from '@kit.AbilityKit';
import { BusinessError } from '@kit.BasicServicesKit';
import { hilog } from '@kit.PerformanceAnalysisKit';
import { window } from '@kit.ArkUI';
import { MaterialKit } from '../common/MaterialKit';

const DOMAIN: number = 0x0000;

export default class EntryAbility extends UIAbility {
  onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    // 本产品以暗色为默认主题（见设计规范 3.4）
    this.context.getApplicationContext().setColorMode(ConfigurationConstant.ColorMode.COLOR_MODE_DARK);
    // 鸿蒙 7 的沉浸光感材质（低版本设备会自动跳过）
    MaterialKit.init();
    hilog.info(DOMAIN, 'TravelChinaFootprint', '%{public}s', 'Ability onCreate');
  }

  onDestroy(): void {
    hilog.info(DOMAIN, 'TravelChinaFootprint', '%{public}s', 'Ability onDestroy');
  }

  onWindowStageCreate(windowStage: window.WindowStage): void {
    windowStage.loadContent('pages/Index', (err) => {
      if (err.code) {
        hilog.error(DOMAIN, 'TravelChinaFootprint', 'Failed to load content: %{public}s', JSON.stringify(err));
        return;
      }
      hilog.info(DOMAIN, 'TravelChinaFootprint', '%{public}s', 'Succeeded in loading the content.');
```

this.enableImmersiveWindow(windowStage);

```
});
}

/**
 * 沉浸式窗口：让地图铺满到状态栏底下，同时把状态栏 / 导航栏的前景色设为白色，
 * 避免出现“黑底黑字”。低版本或异常情况只记录日志，不影响主流程。
 */
private enableImmersiveWindow(windowStage: window.WindowStage): void {
  try {
    const mainWindow: window.Window = windowStage.getMainWindowSync();
    mainWindow.setWindowLayoutFullScreen(true).catch((err: BusinessError) => {
      hilog.warn(DOMAIN, 'TravelChinaFootprint', 'setWindowLayoutFullScreen failed: %{public}s',
        JSON.stringify(err));
    });
    mainWindow.setWindowSystemBarProperties({
      statusBarContentColor: '#FFFFFF',
      navigationBarContentColor: '#FFFFFF'
    }).catch((err: BusinessError) => {
      hilog.warn(DOMAIN, 'TravelChinaFootprint', 'setWindowSystemBarProperties failed: %{public}s',
        JSON.stringify(err));
    });
  }
}
```

```

});
} catch (err) {
hilog.warn(DOMAIN, 'TravelChinaFootprint', 'immersive window unavailable: ${public}s', JSON.stringify(err));
}
}

onWindowStageDestroy(): void {
hilog.info(DOMAIN, 'TravelChinaFootprint', '${public}s', 'Ability onWindowStageDestroy');
}

onForeground(): void {
hilog.info(DOMAIN, 'TravelChinaFootprint', '${public}s', 'Ability onForeground');
}

onBackground(): void {
hilog.info(DOMAIN, 'TravelChinaFootprint', '${public}s', 'Ability onBackground');
}
}

// ===== 文件: pages/Index.ets =====
import { common } from '@kit.AbilityKit';
import { BusinessError } from '@kit.BasicServicesKit';
import { hilog } from '@kit.PerformanceAnalysisKit';
import uiMaterial from '@ohos.arkui.uiMaterial';
import { curves } from '@kit.ArkUI';
import { DatabaseHelper } from '../repository/DatabaseHelper';
import { footprintStore } from '../viewmodel/FootprintStore';
import { AccountService } from '../service/AccountService';
import { MembershipService } from '../service/MembershipService';
import { PaywallVisibilityListener, paywallStore } from '../viewmodel/PaywallStore';
import { PaywallSheet } from '../view/PaywallSheet';
import { Dimens, Motion, Palette } from '../common/Theme';
import { MaterialKit } from '../common/MaterialKit';
import { WindowService } from '../common/WindowService';
import { PrivacyConsent } from '../common/PrivacyConsent';
import { MapPage } from '../view/MapPage';
import { CityListPage } from '../view/CityListPage';
import { StatsPage } from '../view/StatsPage';
import { SettingsPage } from '../view/SettingsPage';
import { PrivacyPolicyView } from '../view/PrivacyPolicy';

const DOMAIN: number = 0x0000;
const TAG: string = 'Index';

/**
 * 底部悬浮页签栏样式。
 *
 * 为什么用 AttributeModifier 而不是直接写在 Tabs 的链式调用里:
 * 1) 沉浸光感的浮起样式 `barFloatingStyle` 是 API 26 (鸿蒙 7) 才有的属性,
 * 本 App 的 compatibleSdkVersion 是 5.0.0(12), 鸿蒙 6 及更早的设备上没有这个方法。

```

```

* 放在这里可以用 try/catch 兜住"方法不存在"的情况，退回旧版"整条页签栏 + 真模糊"。
* 2) `barBottomMargin` 要等窗口避让区测量完才知道（手势条高度因机型而异），
* 用属性修饰器可以在测完之后重新 new 一个，让 ArkUI 重新应用一次样式。
*
* 官方《组件适配沉浸光感》给出的三个硬性条件（缺一不可）：
* barOverlap(true) + vertical(false) + barPosition(BarPosition.End)
* 只有三者同时满足，`systemMaterial` 才会真的被系统渲染出来。
*/
class FloatingTabBarModifier implements AttributeModifier<TabsAttribute> {
private bottomMargin: number = 12;
private material: uiMaterial.ImmersiveMaterial | undefined = undefined;

constructor(bottomMargin: number, material: uiMaterial.ImmersiveMaterial | undefined) {
this.bottomMargin = bottomMargin;
this.material = material;
}

applyNormalAttribute(instance: TabsAttribute): void {
try {
instance.barFloatingStyle({
// 左右留 16vp，页签栏成为一条悬浮胶囊（宽度 = 屏宽 - 32）
barSideMargin: 16,
// 距容器底部：手势条高度 + 8vp 呼吸位
barBottomMargin: this.bottomMargin,
// 官方设计规范：底部悬浮 = THIN 材质 + 渐变颜色蒙层。
// 蒙层从底部往上 140vp 由深到透明，负责"压住"地图，材质负责通透和折射。
maskHeight: 140,
maskColor: '#47070B14',
systemMaterial: this.material
});
} catch (err) {
// 鸿蒙 6 及更早：没有 barFloatingStyle (API 26 新增)，
// 退回"页签栏铺满 + 系统组件级模糊"，视觉上仍是深色毛玻璃。
hilog.warn(DOMAIN, TAG, 'barFloatingStyle unavailable, fallback: %{public}s', JSON.stringify(err));
instance.barBackgroundColor('#B80C1320');
instance.barBackgroundBlurStyle(BlurStyle.COMPONENT_ULTRA_THICK, { colorMode: ThemeColorMode.DARK });
}
}

/**
* 应用主框架：底部 4 个页签（地图 / 清单 / 统计 / 我的）。
*
* 鸿蒙 7 适配要点：
* 1) 底部导航是**系统 Tabs 的悬浮页签栏** (barFloatingStyle + uiMaterial 沉浸光感)，
* 不是自绘导航。原因见 FloatingTabBarModifier 的注释：官方规定只有底部 TabBar
* 和 Navigation 标题栏这两个位置，普通组件的材质才会被渲染；
* 自己拿 Button 拼一条导航，系统只会打一条 "Material inactive: out of scope"。
* 2) 图标用矢量 SVG 交给 BottomTabBarStyle.iconStyle 着色，选中金色、未选中低饱和灰；
* 3) 页签栏浮在内容之上 (barOverlap)，底图从玻璃后面透出来被糊掉，

```

```

* 这就是"沉浸光感"里底部悬浮那一档的标准形态。
*/
@Entry
@ComponentV2
struct Index {
  @Local currentIndex: number = 0;
  @Local loading: boolean = true;
  @Local loadError: string = '';
  /** 底部手势条高度 (vp) , 悬浮导航要往上让出这么多, 避免被系统手势条压住 */
  @Local safeBottom: number = 0;
  /** 页签栏样式修饰器, 窗口避让区测完后重新构造一次 */
  @Local barModifier: AttributeModifier<TabsAttribute> = new FloatingTabBarModifier(12, undefined);
  /** 是否已经读过本机的隐私政策同意状态 (读完之前不渲染主界面, 避免闪一下) */
  @Local consentChecked: boolean = false;
  /** 是否已同意隐私政策; 未同意时主界面之上盖一层同意弹层 */
  @Local consentGiven: boolean = false;
  /**
   * 购买面板是否展示。
   *
   * 这里必须由**主框架**统一承载: 地图页 / 清单页 / 我的页在 Tabs 里同时活着,
   * 如果三个页面各挂一个 bindSheet, 一次 open() 会同时弹出多份面板
   * (实测就是"要点两次关闭"的那个 bug) 。
   */
  @Local paywallShown: boolean = false;
  private paywallListener: PaywallVisibilityListener = (visible: boolean) => {
    this.paywallShown = visible;
  };
  private tabsController: TabsController = new TabsController();

  aboutToAppear(): void {
    // 再试一次: Ability.onCreate 阶段窗口还没就绪, 材质能力探测可能返回 false,
    // 这里 (UI 上下文已可用) 重试, 成功后底部页签栏的材质才会真的渲染出来。
    MaterialKit.init();
    paywallStore.subscribe(this.paywallListener);
    this.start();
  }

  aboutToDisappear(): void {
    paywallStore.unsubscribe(this.paywallListener);
  }

  /**
   * 启动顺序: 先读隐私政策同意状态, 再初始化数据。
   * 官方与华为应用市场审核都要求"征得同意后再收集个人信息",
   * 所以这一步必须排在数据库、定位、账号之前。
   */
  private async start(): Promise<void> {
    try {
      const context = this.getUIContext().getHostContext() as common.UIAbilityContext;
      await PrivacyConsent.init(context);
    }
  }
}

```

```

this.consentGiven = PrivacyConsent.isAgreed();
} catch (err) {
this.consentGiven = false;
hilog.warn(DOMAIN, TAG, 'consent check failed: ${public}s', JSON.stringify(err));
}
this.consentChecked = true;
await this.bootstrap();
}

/** 启动引导: 打开数据库 → 加载城市字典 → 读取本地足迹 */
private async bootstrap(): Promise<void> {
try {
const context = this.getUIContext().getHostContext() as common.UIAbilityContext;
await DatabaseHelper.init(context);
await footprintStore.init(context);
await AccountService.init(context);
await MembershipService.init(context);
// 老用户迁移: 升级到付费版之前点亮的城市不回滚, 只是把额度记满
await MembershipService.ensureMigration(footprintStore.visitedCodes);
paywallStore.attach(context);
// 官方《权益发放》要求: 单机应用启动时查一次已购非消耗型商品,
// 覆盖"卸载重装 / 换新手机"。静默执行, 不阻塞界面, 也不弹任何提示。
paywallStore.restoreOnLaunch();
await WindowService.measure(context);
this.safeBottom = WindowService.safeBottom;
// 材质对象要等 MaterialKit.init() 之后再取; 这里也量到了手势条高度, 重新构造一次修饰器
this.barModifier = new FloatingTabBarModifier(8 + this.safeBottom, MaterialKit.floatingTabBar());
this.loading = false;
hilog.info(DOMAIN, TAG, 'bootstrap done: safeBottom=${public}d material=${public}s',
this.safeBottom, MaterialKit.diagnostics);
} catch (err) {
const error: BusinessError = err as BusinessError;
this.loadError = `初始化失败: ${error.message}`;
this.loading = false;
hilog.error(DOMAIN, TAG, 'bootstrap failed: ${public}s', JSON.stringify(err));
}
}

build() {
Stack({ alignContent: Alignment.Bottom }) {
if (!this.consentChecked || this.loading) {
this.splash('正在准备地图数据...', true)
} else if (this.loadError.length > 0) {
this.splash(this.loadError, false)
} else {
Stack() {
// 背景层: 极夜渐变 + 两处柔光。
// 它有两个作用: 1) 给非地图页一个不空的底; 2) 让上面的沉浸光感材质"有东西可折射",
// 否则纯色背景上的玻璃材质看起来和普通半透明没区别。
Column()

```

```

.width('100%')
.height('100%')
.linearGradient({
angle: 170,
colors: [['#121C31', 0.0], ['#070B14', 0.55], ['#0A1120', 1.0]]
})
// 背景层要铺到状态栏和导航条底下（官方“组件安全区方案”：背景类元素用 expandSafeArea 延伸绘制）
.expandSafeArea([SafeAreaType.SYSTEM], [SafeAreaEdge.TOP, SafeAreaEdge.BOTTOM])
// 购买面板挂在背景层（常驻、不参与内容重建）：
// 整个 App 只此一份，切页签也不会多弹一个。
.bindSheet(this.paywallShown, this.paywallSheet(), {
height: SheetSize.MEDIUM,
dragBar: true,
backgroundColor: Palette.surface,
onDisappear: () => {
this.paywallShown = false;
paywallStore.close();
}
})

Column()
.width(300)
.height(300)
.borderRadius(150)
.backgroundColor('#1FF5C542')
.blur(90)
.position({ x: -110, y: -70 })

Column()
.width(340)
.height(340)
.borderRadius(170)
.backgroundColor('#1422D3EE')
.blur(100)
.position({ x: 180, y: 420 })

// 页签栏的渐变蒙层、圆角、材质全部由 barFloatingStyle 负责；
// 这里不再自己叠“底部渐变遮罩”——自绘的那层会画在页签栏之上，把材质糊住。
Tabs({ barPosition: BarPosition.End, index: this.currentIndex, controller: this.tabsController }) {
TabContent() {
// 传入“当前是否停留在本页”：点位闪烁只在可见时跑
MapPage({ active: this.currentIndex === 0 })
}
.tabBar(this.navTab($r('app.media.ic_nav_map'), '地图'))

TabContent() {
CityListPage()
}
.tabBar(this.navTab($r('app.media.ic_nav_list'), '清单'))

```

```

TabContent() {
// 传入"当前是否停留在统计页": 成就的新解锁检测要在每次进入时做一遍
StatsPage({ active: this.currentIndex === 2 })
}
.tabBar(this.navTab($r('app.media.ic_nav_chart'), '统计'))

TabContent() {
SettingsPage()
}
.tabBar(this.navTab($r('app.media.ic_nav_user'), '我的'))
}
// vertical=false + barPosition=End: 页签栏在容器底部（沉浸光感的必要条件之一）
.vertical(false)
// barOverlap=true: 页签栏浮在内容之上，而不是把内容顶上去（必要条件之二）
.barOverlap(true)
// 切换动画三件套（官方《使用动画》+ Tabs 参考）：
// 1) CONTENT_FIRST_WITH_JUMP: 先把目标页加载好，再"无动画跳到目标页附近、最后一段做动画"。
// 这是关键的体验修复——默认动画会按页签序号差横向滑动，
// 从"地图"(0) 直接切到"我的"(3) 时内容要从三屏外飞进来，看着很生硬；
// 开了这个模式后，长距离跳转只播最后一段，观感和其他切换一致。
// 2) 用弹簧曲线代替默认的线性缓动，收尾有自然的"落定感"。
// 3) 时长略长于默认，给弹簧留出回弹空间。
.animationMode(AnimationMode.CONTENT_FIRST_WITH_JUMP)
.animationDuration(Motion.pageSwitch)
.animationCurve(curves.springMotion(0.42, 0.9))
// 悬浮样式 + 沉浸光感材质（必要条件之三，见 FloatingTabBarModifier）
.attributeModifier(this.barModifier)
.width('100%')
.height('100%')
.onChange((index: number) => {
this.currentIndex = index;
})
}
.width('100%')
.height('100%')

// 首次启动的隐私政策同意弹层：盖在主界面之上，不同意就退出应用。
// 说明：这是国内应用市场审核的硬性要求（显著提示 + 征得同意）。
if (!this.consentGiven) {
this.privacyGate()
}
}
}
.width('100%')
.height('100%')
.backgroundColor(Palette.bg)
}

/** 全局唯一的购买面板（见 paywallShown 的说明） */
@Builder

```

```
paywallSheet() {
  PaywallSheet()
}

@Builder
privacyGate() {
  Column() {
    Column({ space: 12 }) {
      Text('欢迎使用「走遍中国」')
        .fontSize(18)
        .fontWeight(FontWeight.Bold)
        .fontColor(Palette.text)
        .width('100%')
      Text('在使用前, 请你阅读并同意我们的隐私政策。我们坚持最小必要原则: '
+ '不注册也能用全部核心功能, 位置与账号都由你主动触发。')
        .fontSize(12)
        .lineHeight(19)
        .fontColor(Palette.textDim)
        .width('100%')

      Column() {
        PrivacyPolicyView()
      }
        .width('100%')
        .layoutWeight(1)
        .padding({ left: 14, right: 14, top: 10, bottom: 10 })
        .borderRadius(14)
        .backgroundColor(Palette.surfaceSoft)
        .border({ width: 0.5, color: Palette.line })

      Row({ space: 12 }) {
        Button('不同意并退出')
          .layoutWeight(1)
          .height(44)
          .borderRadius(22)
          .fontSize(14)
          .backgroundColor(Palette.surface)
          .fontColor(Palette.textDim)
          .onClick(() => {
            this.exitApp();
          })
        Button('同意并继续')
          .layoutWeight(1)
          .height(44)
          .borderRadius(22)
          .fontSize(14)
          .fontWeight(FontWeight.Medium)
          .backgroundColor(Palette.gold)
          .fontColor('#1A1405')
          .onClick(() => {
```

```

this.acceptPrivacy();
})
}
.width('100%')
}
.width('100%')
.height('78%')
.padding(20)
.borderRadius(24)
.backgroundColor(Palette.surface)
.border({ width: 0.5, color: Palette.line })
.shadow({ radius: 30, color: '#B3000000', offsetX: 0, offsetY: 12 })
}
.width('100%')
.height('100%')
.padding({ left: 20, right: 20 })
.justifyContent(FlexAlign.Center)
.backgroundColor('#D9070B14')
}

private async acceptPrivacy(): Promise<void> {
await PrivacyConsent.agree();
this.consentGiven = true;
}

/** 不同意: 按审核要求, 不提供任何服务并退出应用 */
private exitApp(): void {
const context = this.getUIContext().getHostContext() as common.UIAbilityContext;
context.terminateSelf().catch((err: BusinessError) => {
hilog.warn(DOMAIN, TAG, 'terminateSelf failed: %{public}s', JSON.stringify(err));
});
}

@Builder
splash(message: string, busy: boolean) {
Column() {
if (busy) {
LoadingProgress()
.width(44)
.height(44)
.color(Palette.gold)
Text(message)
.fontSize(13)
.fontColor(Palette.textDim)
.margin({ top: 14 })
} else {
Text(message)
.fontSize(13)
.fontColor(Palette.textDim)
.textAlign(TextAlign.Center)

```

```

Button('重试')
  .width(128)
  .height(42)
  .borderRadius(21)
  .backgroundColor(Palette.gold)
  .fontColor('#1A1405')
  .fontSize(14)
  .margin({ top: 18 })
  .onClick(() => {
    this.loading = true;
    this.loadError = '';
    this.bootstrap();
  })
}
}

.width('100%')
.height('100%')
.justifyContent(FlexAlign.Center)
.padding(24)
}

/**
 * 页签样式：图标 + 文字，选中金色、未选中灰。
 *
 * 高亮胶囊、按压反馈、图标与文字的排布全部交给系统绘制
 * （官方《组件适配沉浸光感》里"底部页签"就是 BottomTabBarStyle 这一条路径），
 * 我们只提供图标资源、文案和配色。
 */
private navTab(icon: Resource, label: string): BottomTabBarStyle {
  return new BottomTabBarStyle(icon, label)
    .labelStyle({
      selectedColor: Palette.goldLight,
      // 玻璃材质上的文字必须是纯白，灰色在浅色底图透上来时会发闷、看不清
      unselectedColor: Palette.text,
      font: { size: 10, weight: FontWeight.Medium }
    })
    .iconStyle({
      selectedColor: Palette.gold,
      unselectedColor: Palette.text
    });
}

// ===== 文件：view/BadgeCard.ets =====
import { curves } from '@kit.ArkUI';
import { Motion, Palette } from '../common/Theme';
import { BadgeCell } from '../viewmodel/BadgeCell';

/**
 * 单枚成就徽章卡片。

```

```

*
* 单独抽成组件的原因见 BadgeCell 的注释:
* 它读取的是 @ObservedV2 对象的 @Trace 字段, 字段一变本组件就刷新,
* 不受父级 ForEach 的复用策略影响一点亮动画才能真的跑起来。
*/
@ComponentV2
export struct BadgeCard {
  @Param cell: BadgeCell = new BadgeCell('', '', '', '');
  /** 在网格里的序号, 用于入场动效错开时间 */
  @Param index: number = 0;
  /** 页面入场标记 (父组件控制) */
  @Param entered: boolean = false;
  /** 卡片宽度: 网格里是 31.5% (一行三个), 试播时可以用别的值 */
  @Param cardWidth: Length = '31.5%';

  build() {
    Stack({ alignContent: Alignment.Center }) {
      Column() {
        // 图标区: 光晕 + 扩散光环 + 图标
        Stack({ alignContent: Alignment.Center }) {
          // 常驻金色光晕 (已解锁才亮); 庆祝时随爆发动画放大淡出
          Column()
            .width(50)
            .height(50)
            .borderRadius(25)
            .radialGradient({
              center: ['50%', '50%'],
              radius: '50%',
              colors: [['#59F5C542', 0.0], ['#2EF5C542', 0.55], ['#00F5C542', 1.0]]
            })
            .opacity(this.cell.celebrating
              ? this.cell.glowOpacity
              : (this.cell.unlocked ? 0.85 : 0))
            .scale({
              x: this.cell.celebrating ? this.cell.glowScale : 1,
              y: this.cell.celebrating ? this.cell.glowScale : 1
            })

          // 两圈依次向外扩散的金色光环
          if (this.cell.celebrating) {
            Column()
              .width(46)
              .height(46)
              .borderRadius(23)
              .border({ width: 2, color: '#E6F5C542' })
              .scale({ x: this.cell.ring1Scale, y: this.cell.ring1Scale })
              .opacity(this.cell.ring1Opacity)
            Column()
              .width(46)
              .height(46)

```

```

.borderRadius(23)
.border({ width: 1.5, color: '#B3FFD97D' })
.scale({ x: this.cell.ring2Scale, y: this.cell.ring2Scale })
.opacity(this.cell.ring2Opacity)
}

// 徽章本体: 升起 + 放大 + 绕 Y 轴转两圈
Text(this.cell.icon)
.fontSize(26)
.scale({
x: this.cell.celebrating ? this.cell.iconScale : 1,
y: this.cell.celebrating ? this.cell.iconScale : 1
})
.translate({
x: this.cell.celebrating ? this.cell.iconShake : 0,
y: this.cell.celebrating ? this.cell.iconLift : 0
})
// 三轴旋转合成: 每种效果只用其中一个轴, 所以按"哪个角度非零"选轴、
// 角度取三者之和即可 (ArkUI 的 rotate 只能给一个轴向量 + 一个角度)
.rotate({
x: this.cell.celebrating && this.cell.iconSpinX !== 0 ? 1 : 0,
y: this.cell.celebrating && this.cell.iconSpin !== 0 ? 1 : 0,
z: this.cell.celebrating && this.cell.iconSpinZ !== 0 ? 1 : 0,
angle: this.cell.celebrating
? this.cell.iconSpin + this.cell.iconSpinX + this.cell.iconSpinZ : 0
})
// 未解锁的徽章用灰度滤镜压下去, 比单纯降透明度更像"还没点亮"
.grayscale(this.cell.unlocked ? 0 : 1)
.opacity(this.cell.celebrating
? this.cell.iconOpacity
: (this.cell.unlocked ? 1 : 0.65))
}
.width(50)
.height(50)

Text(this.cell.name)
.fontSize(13)
.fontColor(this.cell.unlocked ? Palette.text : Palette.textDim)
.fontWeight(this.cell.unlocked ? FontWeight.Medium : FontWeight.Normal)
.margin({ top: 6 })
.maxLines(1)
Text(this.cell.desc)
.fontSize(10)
.fontColor(Palette.textDim)
.margin({ top: 2 })
.maxLines(2)
.textOverflow({ overflow: TextOverflow.Ellipsis })
.textAlign(TextAlign.Center)
Text(this.cell.unlocked ? ' 已解锁' : `${this.cell.progress}/${this.cell.target}`)
.fontSize(10)

```

```

.fontColor(this.cell.unlocked ? Palette.gold : Palette.textDim)
.fontWeight(this.cell.unlocked ? FontWeight.Medium : FontWeight.Normal)
.margin({ top: 6 })
}
.width('100%')
.height('100%')
.justifyContent(FlexAlign.Center)
.padding(8)

// 扫光: 一条斜向白光从卡片左侧扫到右侧 (只在庆祝时出现)
if (this.cell.celebrating) {
Column()
.width(26)
.height(220)
.linearGradient({
angle: 18,
colors: [['#00FFFFFF', 0.0], ['#80FFFFFF', 0.5], ['#00FFFFFF', 1.0]]
})
.translate({ x: this.cell.shineX })
.rotate({ angle: 16 })
.hitTestBehavior(HitTestMode.None)
}
}
// 三列布局: 每格占比 31.5%, 配合 SpaceBetween 正好一行三个
.width(this.cardWidth)
.height(122)
.margin({ bottom: 10 })
.borderRadius(16)
.clip(true)
.backgroundColor(this.cell.unlocked ? '#33F5C542' : '#1A0F1626')
.backgroundBlurStyle(BlurStyle.COMPONENT_REGULAR, { colorMode: ThemeColorMode.DARK })
.border({
width: this.cell.celebrating ? 1.6 : (this.cell.unlocked ? 0.8 : 0.5),
color: this.cell.unlocked ? '#66F5C542' : '#14FFFFFF'
})
// 徽章浮起时会越过卡片上沿, 抬高层级避免被上一行的卡片压住
.zIndex(this.cell.celebrating ? 3 : 1)
// 庆祝时卡片自身弹起一下
.scale({
x: this.cell.celebrating ? this.cell.cardScale : 1,
y: this.cell.celebrating ? this.cell.cardScale : 1
})
.rotate({ angle: this.cell.celebrating ? this.cell.cardRotate : 0 })
// 入场: 逐枚错开淡入上浮, 一行三个像"依次落位"
.opacity(this.entered ? 1 : 0)
.translate({ y: this.entered ? 0 : 12 })
.animation({
duration: Motion.normal + 60,
delay: 40 + this.index * 45,
curve: curves.springMotion(0.42, 0.9)
})

```

```
  })
}
}

// ===== 文件: view/CelebrationBannerView.ets =====
import { Palette } from '../common/Theme';
import { CelebrationBanner } from '../viewmodel/CelebrationBanner';

/**
 * 顶部"成就解锁"横幅: 金色渐变卡片 + 流光扫过。
 * 位置固定贴在顶部 (用 translate 做滑入滑出, 不触发重新布局)。
 */
@ComponentV2
export struct CelebrationBannerView {
  @Param banner: CelebrationBanner = new CelebrationBanner();
  /** 状态栏高度 (vp), 横幅要避免 */
  @Param topInset: number = 0;

  build() {
    Row({ space: 12 }) {
      // 徽章图标
      Text(this.banner.icon)
        .fontSize(30)

      Column({ space: 2 }) {
        Text(this.banner.title)
          .fontSize(15)
          .fontWeight(FontWeight.Medium)
          .fontColor('#3A2A05')
        Text(this.banner.subtitle)
          .fontSize(11)
          .fontColor('#6B5313')
          .maxLines(1)
          .textOverflow({ overflow: TextOverflow.Ellipsis })
      }
      .alignItems(HorizontalAlign.Start)
      .layoutWeight(1)

      Text('NEW')
        .fontSize(10)
        .fontWeight(FontWeight.Bold)
        .fontColor('#3A2A05')
        .padding({ left: 8, right: 8, top: 3, bottom: 3 })
        .borderRadius(999)
        .backgroundColor('#66FFFFFF')
    }
    .width('100%')
    .padding({ left: 16, right: 16, top: 12, bottom: 12 })
    .borderRadius(18)
    .linearGradient({
```

```

angle: 100,
colors: [['#FFE58A', 0.0], ['#F5C542', 0.55], ['#E0A81F', 1.0]]
})
// 流光层: 盖在横幅上的一条斜向白光
.overlay(this.shimmer(), { align: Alignment.Center })
.shadow({ radius: 24, color: '#8CF5C542', offsetX: 0, offsetY: 10 })
.translate({ y: this.banner.offsetY })
.scale({ x: this.banner.scale, y: this.banner.scale })
.opacity(this.banner.opacity)
.hitTestBehavior(HitTestMode.None)
.position({ x: 0, y: this.topInset + 8 })
}

```

```

@Builder
shimmer() {
Column() {
Column()
.width(38)
.height(200)
.linearGradient({
angle: 18,
colors: [['#00FFFFFF', 0.0], ['#59FFFFFF', 0.5], ['#00FFFFFF', 1.0]]
})
.translate({ x: this.banner.shimmerX })
.rotate({ angle: 16 })
}
.width('100%')
.height('100%')
.borderRadius(18)
.clip(true)
}
}

```

```

// ===== 文件: view/CityListPage.ets =====
import { curves, promptAction } from '@kit.ArkUI';
import { hilog } from '@kit.PerformanceAnalysisKit';
import { CityItem } from '../model/CityDict';
import { CityRepository } from '../repository/CityRepository';
import { footprintStore } from '../viewmodel/FootprintStore';
import { LightBatchResult } from '../viewmodel/FootprintStore';
import { paywallStore } from '../viewmodel/PaywallStore';
import { Dimens, Motion, Palette } from '../common/Theme';
import { WindowService } from '../common/WindowService';

```

```

/**
 * 足迹清单 (P-07) 。
 * 已点亮 / 未点亮两个视图, 支持搜索; 点右侧按钮直接点亮或取消。
 */

```

```

@ComponentV2
export struct CityListPage {

```

```
@Local showVisited: boolean = true;
@Local keyword: string = '';
/** 批量点亮模式 */
@Local batchMode: boolean = false;
/** 批量模式下已勾选的城市 */
@Local selected: string[] = [];
@Local submitting: boolean = false;
/** 沉浸式布局下避开状态栏 (vp) */
@Local safeTop: number = 0;
/** 底部导航条高度 (vp) , 滚动内容要额外让出这么多 */
@Local safeBottom: number = 0;
/** 入场动效标记: 首次进入本页时由 false → true, 配合 .animation 做淡入上浮 */
@Local entered: boolean = false;

aboutToAppear(): void {
  this.safeTop = WindowService.safeTop;
  this.safeBottom = WindowService.safeBottom;
  this.playEnter();
}

/**
 * 首次进入本页时的入场动效: 淡入 + 轻微上浮。
 * 用 setTimeout 让初值先渲染一帧, 再翻转让 .animation 捕捉到属性变化
 * (官方《使用动画》里"属性动画"就是这么用的: 属性变化 → 动画跟随)。
 */
private playEnter(): void {
  this.entered = false;
  setTimeout(() => {
    this.entered = true;
  }, 30);
}

private isSelected(code: string): boolean {
  return this.selected.includes(code);
}

private toggleSelect(code: string): void {
  if (this.isSelected(code)) {
    this.selected = this.selected.filter((item: string) => item !== code);
  } else {
    const next: string[] = this.selected.slice();
    next.push(code);
    this.selected = next;
  }
}

private async submitBatch(): Promise<void> {
  if (this.submitting || this.selected.length === 0) {
    return;
  }
}
```

```

this.submitting = true;
try {
const result: LightBatchResult = await footprintStore.lightUpBatch(this.selected);
this.selected = [];
this.batchMode = false;
// 免费额度不够的部分: 引导开通会员, 开通后由页面的批量入口重新提交
if (result.blocked > 0) {
paywallStore.onPurchased = undefined;
paywallStore.open('');
// 注意: 这里**不能**用 Toast 汇报结果。
// 实测 Toast 的浮层会把刚刚打开的半模态顶掉 (面板不显示),
// 所以把结果写进面板本身, 信息还更完整。
paywallStore.statusText =
`已点亮 ${result.lit} 座, 还有 ${result.blocked} 座需要开通后才能点亮`;
return;
}
if (result.lit > 0) {
promptAction.showToast({ message: `已点亮 ${result.lit} 座城市` });
}
} catch (err) {
hilog.error(0x0000, 'CityListPage', 'batch light failed: %{public}s', JSON.stringify(err));
promptAction.showToast({ message: '批量点亮失败, 请重试' });
} finally {
this.submitting = false;
}
}

private currentList(): CityItem[] {
const matched: CityItem[] = CityRepository.search(this.keyword);
const result: CityItem[] = [];
for (const city of matched) {
const visited: boolean = footprintStore.isVisited(city.code);
if (visited === this.showVisited) {
result.push(city);
}
}
return result;
}

private async toggle(city: CityItem): Promise<void> {
if (footprintStore.isVisited(city.code)) {
await footprintStore.unlight(city.code);
promptAction.showToast({ message: `已取消点亮 ${city.name}` });
} else {
const ok: boolean = await footprintStore.lightUp(city.code);
if (ok) {
promptAction.showToast({ message: `已点亮 ${city.name}` });
} else {
this.requestMembership(city);
}
}
}

```

```

}
}

/** 免费额度用完：先弹购买面板，开通成功后自动把这座城市点亮 */
private requestMembership(city: CityItem): void {
  paywallStore.onPurchased = () => {
    footprintStore.lightUp(city.code).then((done: boolean) => {
      if (done) {
        promptAction.showToast({ message: `已点亮 ${city.name}` });
      }
    });
  };
  paywallStore.open(city.name);
}

build() {
  Stack({ alignContent: Alignment.TopStart }) {
    Column() {
      // 顶部：标题 + 分段 + 搜索（玻璃卡片）。
      // 左右留边用外层 padding，不用 width('100%') + margin（后者会让右侧溢出屏幕）。
      Column() {
        Column() {
          Row() {
            Column() {
              Text('足迹清单')
                .fontSize(22)
                .fontColor(Palette.text)
                .fontWeight(FontWeight.Medium)
              Text(`${footprintStore.visitedCount} / ${footprintStore.totalUnits} 个市州级行政区`)
                .fontSize(11)
                .fontColor(Palette.textDim)
                .margin({ top: 4 })
            }
            .alignItems(HorizontalAlign.Start)
            .layoutWeight(1)

            Text(this.batchMode ? '退出批量' : '批量点亮')
              .fontSize(12)
              .fontColor(this.batchMode ? Palette.text : Palette.gold)
              .padding({ left: 14, right: 14, top: 8, bottom: 8 })
              .borderRadius(999)
              .backgroundColor('#1FF5C542')
              .border({ width: 0.5, color: '#4DF5C542' })
              .onClick(() => {
                this.batchMode = !this.batchMode;
                if (!this.batchMode) {
                  this.selected = [];
                }
              })

```

```
}  
.width('100%')  
.alignItems(VerticalAlign.Center)  
  
Row() {  
  // 注意: 这里不能用 @Builder 传字符串 (ArkUI 的 @Builder 按值传参不参与刷新,  
  // 会出现"点亮了城市但分段数字还是 0"的问题), 必须让 Text 直接读状态。  
  Text('已点亮 ' + footprintStore.visitedCount)  
    .fontSize(13)  
    .fontColor(this.showVisited ? Palette.text : Palette.textDim)  
    .fontWeight(this.showVisited ? FontWeight.Medium : FontWeight.Normal)  
    .textAlign(TextAlign.Center)  
    .layoutWeight(1)  
    .padding({ top: 10, bottom: 10 })  
    .borderRadius(Dimens.pillRadius)  
    .backgroundColor(this.showVisited ? '#33F5C542' : Color.Transparent)  
    .animation({ duration: Motion.normal, curve: Curve.EaseOut })  
    .onClick(() => {  
      this.showVisited = true;  
    })  
  
  Text('未点亮 ' + (footprintStore.totalUnits - footprintStore.visitedCount))  
    .fontSize(13)  
    .fontColor(this.showVisited ? Palette.textDim : Palette.text)  
    .fontWeight(this.showVisited ? FontWeight.Normal : FontWeight.Medium)  
    .textAlign(TextAlign.Center)  
    .layoutWeight(1)  
    .padding({ top: 10, bottom: 10 })  
    .borderRadius(Dimens.pillRadius)  
    .backgroundColor(this.showVisited ? Color.Transparent : '#33F5C542')  
    .animation({ duration: Motion.normal, curve: Curve.EaseOut })  
    .onClick(() => {  
      this.showVisited = false;  
    })  
}  
  .width('100%')  
  .padding(3)  
  .borderRadius(Dimens.pillRadius)  
  .backgroundColor('#66070B14')  
  .margin({ top: 14 })  
  
  TextInput({ placeholder: '搜索城市、省份', text: this.keyword })  
    .height(44)  
    .borderRadius(14)  
    .backgroundColor('#66070B14')  
    .placeholderColor(Palette.textDim)  
    .fontColor(Palette.text)  
    .margin({ top: 10 })  
    .onChange((value: string) => {  
      this.keyword = value;  
    })  
}
```

```

}))
}
.width('100%')
.padding(16)
.borderRadius(Dimens.cardRadius)
.backgroundColor('#26070B14')
.backgroundBlurStyle(BlurStyle.COMPONENT_THICK, { colorMode: ThemeColorMode.DARK })
.border({ width: 0.5, color: '#1FFFFFFF' })
}
.width('100%')
.padding({ left: 16, right: 16, top: 12 + this.safeTop, bottom: 10 })

List({ space: 8 }) {
  ForEach(this.currentList(), (city: CityItem) => {
    ListItem() {
      Row() {
        // 状态指示条: 已点亮是金色, 未点亮是灰蓝
        Column()
          .width(3)
          .height(34)
          .borderRadius(2)
          .backgroundColor(footprintStore.isVisited(city.code) ? Palette.gold : Palette.unlitStroke)
          .margin({ right: 12 })

        Column() {
          Text(city.name)
            .fontSize(15)
            .fontColor(Palette.text)
          Text(`${city.parent} · ${city.code}`)
            .fontSize(11)
            .fontColor(Palette.textDim)
          .margin({ top: 3 })
        }
        .alignItems(HorizontalAlign.Start)
        .layoutWeight(1)

        if (this.batchMode) {
          Text(this.isSelected(city.code) ? ' ' : 'o')
            .fontSize(16)
            .fontColor(this.isSelected(city.code) ? Palette.gold : Palette.textDim)
            .width(28)
            .textAlign(TextAlign.Center)
        } else {
          Text(footprintStore.isVisited(city.code) ? '已点亮' : '点亮')
            .fontSize(13)
            .fontColor(footprintStore.isVisited(city.code) ? Palette.gold : Palette.textDim)
            .padding({ left: 14, right: 14, top: 7, bottom: 7 })
            .borderRadius(999)
            .backgroundColor(Palette.surfaceSoft)
            .onClick(() => {

```

```

this.toggle(city);
})
}
}
.width('100%')
.padding(14)
.borderRadius(16)
.backgroundColor('#26070B14')
.backgroundBlurStyle(BlurStyle.COMPONENT_REGULAR, { colorMode: ThemeColorMode.DARK })
.border({ width: 0.5, color: '#14FFFFFF' })
.alignItems(VerticalAlign.Center)
.onClick(() => {
if (this.batchMode && !footprintStore.isVisited(city.code)) {
this.toggleSelect(city.code);
}
})
}
// 同 StatsPage: key 要带上点亮状态, 否则取消/点亮后行内容不会刷新
}, (city: CityItem) => `${city.code}_${footprintStore.isVisited(city.code) ? 1 : 0}`)
}
.width('100%')
.layoutWeight(1)
.padding({ left: 16, right: 16 })
.scrollBar(BarState.Off)
.contentEndOffset(Dimens.navSpace + this.safeBottom)

if (this.currentList().length === 0) {
Text(this.showVisited ? '还没有点亮的城市, 去地图上点一个吧' : '全部城市都已点亮')
.fontSize(13)
.fontColor(Palette.textDim)
// 底部要同时让开悬浮导航和手势条, 否则最后一行会被压住
.margin({ bottom: Dimens.navSpace + this.safeBottom })
}

// 批量模式底部操作条
if (this.batchMode) {
Row() {
Column() {
Text(`已选 ${this.selected.length} 个城市`)
.fontSize(13)
.fontColor(Palette.text)
Text('同一次点亮会记录为今天到访')
.fontSize(10)
.fontColor(Palette.textDim)
.margin({ top: 2 })
}
.alignItems(HorizontalAlign.Start)
.layoutWeight(1)

Button(this.submitting ? '处理中...' : '点亮')

```

```

.width(112)
.height(40)
.borderRadius(20)
.fontSize(14)
.backgroundColor(this.selected.length > 0 ? Palette.gold : Palette.line)
.fontColor(this.selected.length > 0 ? '#1A1405' : Palette.textDim)
.enabled(this.selected.length > 0 && !this.submitting)
.onClick(() => {
  this.submitBatch();
})
}
.width('100%')
.padding({ left: 16, right: 16, top: 12, bottom: 12 })
.margin({ left: 16, right: 16, bottom: Dimens.navSpace + this.safeBottom - 14 })
.borderRadius(Dimens.cardRadius)
.backgroundColor('#B80C1320')
.backgroundBlurStyle(BlurStyle.COMPONENT_ULTRA_THICK, { colorMode: ThemeColorMode.DARK })
.border({ width: 0.5, color: '#1FFFFFFF' })
}
}
.width('100%')
.height('100%')
.backgroundColor(Palette.bg)

// 顶部渐隐遮罩：滚动内容接近状态栏时淡出，不与系统时间/信号图标打架
Column()
.width('100%')
.height(this.safeTop + 12)
.linearGradient({
  angle: 180,
  colors: [['#FF070B14', 0.0], ['#F2070B14', 0.7], ['#00070B14', 1.0]]
})
.hitTestBehavior(HitTestMode.None)
.position({ x: 0, y: 0 })
}
.width('100%')
.height('100%')
.backgroundColor(Palette.bg)
.opacity(this.entered ? 1 : 0)
.translate({ y: this.entered ? 0 : 14 })
.animation({ duration: Motion.normal + 80, curve: curves.springMotion(0.42, 0.9) })
}

}

// ===== 文件: view/MapPage.ets =====
import { AsyncCallback, BusinessError } from '@kit.BasicServicesKit';
import { MapComponent, mapCommon, map } from '@kit.MapKit';
import { curves } from '@kit.ArkUI';
import { hilog } from '@kit.PerformanceAnalysisKit';

```

```

import { promptAction } from '@kit.ArkUI';
import { common } from '@kit.AbilityKit';
import { CityItem, CountryItem, ProvinceItem } from '../model/CityDict';
import { CityRepository } from '../repository/CityRepository';
import { FootprintListener, footprintStore } from '../viewmodel/FootprintStore';
import { LocationService } from '../service/LocationService';
import { Dimens, MapDefaults, MapLabelStyle, Motion, Palette } from '../common/Theme';
import { LitColors, LitColorPreset } from '../viewmodel/LitColors';
import { AchievementEngine } from '../viewmodel/AchievementEngine';
import { FREE_QUOTA, membershipState } from '../service/MembershipService';
import { paywallStore } from '../viewmodel/PaywallStore';
import { WindowService } from '../common/WindowService';

const DOMAIN: number = 0x0000;
const TAG: string = 'MapPage';

/** 打卡标签选项 */
interface TagOption {
  value: string;
  label: string;
}

/** 地图上的城市名标注（由经纬度投影到屏幕坐标后绘制） */
interface CityLabel {
  code: string;
  name: string;
  /** 标签中心的屏幕坐标（vp）。城市标签的 y 已含下移偏移，不再是点位本身 */
  x: number;
  y: number;
  /** 该标签"本来应该在"的锚点位置（vp）：被挤开时用它画虚线牵引线指回去 */
  anchorX: number;
  anchorY: number;
  /** 是否被挤离锚点（true 时画一条虚线从锚点连到标签） */
  leader: boolean;
  visited: boolean;
  /** 'province' = 省名（大一号、无底色）；'city' = 地级市名 */
  kind: string;
  /** 标签尺寸（vp）：由字数与字号估算，用于避让排布和精确居中 */
  w: number;
  h: number;
  /** 排布优先级，数字越小越优先占位 */
  priority: number;
}

/** 城市标签相对点位的下移距离（vp），避免压住圆点 */
const CITY_LABEL_OFFSET: number = 9;
/** 标签之间的最小间隙（vp） */
const LABEL_GAP: number = 2;
/**
 * 省名允许的**字形**重叠比例（不是包围盒）。

```

```

* 全国视野下东部省份挨得很近，若要求"零重叠"，山东、江西这些会被整片挤掉；
* 放到 40% 后：山东(与河北≈11%)、江西(与湖南≈33%) 能显示，
* 而天津(与北京≈49%)、香港/澳门(与广东≈48~52%) 仍会自动隐藏—
* 正好符合"位置必须准确，挤不下的宁可不显示"的规则。
*/
const PROVINCE_OVERLAP_TOLERANCE: number = 0.25;
/** 与底图国名允许的字形重叠（几乎为 0，只是留一点像素级余量） */
const RESERVED_OVERLAP_TOLERANCE: number = 0.1;
/**
* 汉字字形相对包围盒的收缩比例。
* 包围盒 = 字形 + 安全留白，实际墨迹只有约 0.80 宽、0.53 高
* (11.9vp 字号：字宽 11.9、字高约 10.5，而包围盒是 29.8×19.9) 。
* 之前取 0.62 偏高，把"其实没压上"的省份（河北 vs 北京）也判成冲突了。
*/
const INK_WIDTH_RATIO: number = 0.8;
const INK_HEIGHT_RATIO: number = 0.53;
/**
* 标注字号 (vp)，**按底图实测值反推**：
* 在真机截图里量底图国名"中华人民共和国"的"中"字，字形高 38px = 10.9vp，
* CJK 字形约占 0.88em → 底图字号 ≈ 12.3vp。
* 因此这里取省名 12、城市名 11（省名略大形成层级），字重统一 Medium，
* 与底图笔画粗细一致，不再出现"我们的字又大又浅"的割裂感。
*/
const PROVINCE_FONT_SIZE: number = 11.9;
const CITY_FONT_SIZE: number = 11.9;

/**
* 国家名（自绘）字号。
* 底图自带的国家名已在 Petal Maps Studio 里整类隐藏——它固定画在中国正中央，
* 会压住河北 / 山西 / 宁夏几个省名，而且我们挪不动它。
* 改由自己绘制后，它和省市名一起参与避让，位置也由我们决定。
*/
const COUNTRY_FONT_SIZE: number = 11.9;

/**
* 全国视野下不标的省级行政区（用户已确认可以隐藏）。
* 天津被北京/河北夹住、香港澳门紧贴广东，在这个缩放级别它们的名字与邻省**必然重叠**
* （实测：天津↔北京字形重叠 44%，香港/澳门↔广东 45%+），与其糊成一团不如不标。
* 放大到 zoom≥4.5 进入省会层后，它们会以"省会/特别行政区"的身份自然出现。
*/
const NATIONAL_SKIP_CODES: string[] = ['120000', '810000', '820000'];

/**
* 自定义地图样式（Petal Maps Studio 发布）。
* 样式内容：隐藏「行政区划 → 主要城市」的标签要素，
* 去掉底图自带的城市点位符号，避免与 App 自己绘制的 370 个城市点位重叠。
* 样式 ID 为发布后的唯一 ID，在 Studio 中修改样式后所有使用该 ID 的应用自动生效。
*/
const MAP_STYLE_ID: string = '2038502036122762880';

```

```

/**
 * 34 个省级行政区的行政中心（省会 / 首府 / 直辖市 / 港澳台）行政区划代码。
 * 用于分级标注：【低缩放】只显示省会 + 底图的省份名；【放大后】再显示全部地级市。
 */
const CAPITAL_CODES: string[] = [
  '110000', '120000', '130100', '140100', '150100', '210100',
  '220100', '230100', '310000', '320100', '330100', '340100',
  '350100', '360100', '370100', '410100', '420100', '430100',
  '440100', '450100', '460100', '500000', '510100', '520100',
  '530100', '540100', '610100', '620100', '630100', '640100',
  '650100', '710000', '810000', '820000'
];

/**
 * 足迹地图（P-02）。
 * 实现路线：华为 Map Kit 底图 + 370 个城市点位（指导书附录 G "点位方案"）。
 * 合规要点：
 * 1) 底图使用华为地图服务，审图号通过 setApproveNumberEnabled 显示；
 * 2) 不自行绘制行政区边界几何，只做标注（属于法规允许的"增值服务-标注"）；
 * 3) 不遮挡地图 Logo。
 */
@ComponentV2
export struct MapPage {
  private mapOptions?: mapCommon.MapOptions;
  private callback?: AsyncCallback<map.MapComponentController>;
  private controller?: map.MapComponentController;
  private markers: Map<string, map.Marker> = new Map<string, map.Marker>();
  private listener: FootprintListener = (code: string, visited: boolean) => {
    this.syncMarker(code, visited);
  }
  // 点亮状态变化会影响城市名的颜色，需要刷新标注
  this.updateCityLabels();
};

@Local mapReady: boolean = false;
@Local hint: string = '地图加载中...';
@Local sheetShown: boolean = false;
@Local selectedCode: string = '';
@Local locating: boolean = false;
@Local showUnvisited: boolean = true;
/** 当前是否处于"能看清细节"的缩放级别（用于点位疏密控制） */
@Local detailZoom: boolean = false;
/** 沉浸式布局下需要避开状态栏 / 手势条高度（vp） */
@Local safeTop: number = 0;
@Local safeBottom: number = 0;
/** 打卡表单：标签与备注 */
@Local selectedTag: string = 'other';
@Local noteInput: string = '';

/** 记录每个点位当前的透明度，避免重复下发原发生指令 */
private markerAlpha: Map<string, number> = new Map<string, number>();

```

```

/** 当前屏幕上的城市名标注 */
@Local cityLabels: CityLabel[] = [];
/** 地图容器尺寸 (vp) , 用于判断标注是否在视野内 */
private mapWidth: number = 0;
private mapHeight: number = 0;
/** 标签刷新重试计数 (容器尺寸未就绪时用) */
private labelRetry: number = 0;
/** 上一次"跟随相机"刷新标注的时间戳 (毫秒) , 用于节流 */
private lastCameraTick: number = 0;
/** 点位闪烁定时器句柄 */
private blinkTimer: number = -1;
/** 每座城市的闪烁周期与相位缓存 (由行政区划代码派生, 稳定不变) */
private blinkParams: Map<string, number[]> = new Map<string, number[]>();
/** 自己识别点按时, 记录按下位置与时刻 */
private touchDownX: number = 0;
private touchDownY: number = 0;
private touchDownAt: number = 0;
/** 本次触摸是否已经移动超过容差 (移动过就不算点击) */
private touchMoved: boolean = false;
/** 上一次处理点按的时刻: 地图自己的 mapClick 与我们的 onTouch 会同时命中同一次点按, 用它去重 */
private lastPickAt: number = 0;
/** TODO 临时诊断: 被挤掉的省名 (地图 SDK 日志刷爆配额, hilog 会被丢弃, 只能画在屏幕上) */
@Local debugDropped: string = '';
/** 当前选中的点亮配色编号 (0~7, 在「我的 → 点亮颜色」里切换) */
@Local litColorIndex: number = 0;
/**
 * 由父组件传入: 当前是否停留在本页。
 * 点位闪烁只在页面可见时跑, 切走就停 (省电), 切回来接着闪。
 */
@Param active: boolean = true;

@Monitor('active')
onActiveChanged(): void {
  if (this.active) {
    this.loadLitColor();
    this.startBlinking();
  } else {
    this.stopBlinking();
  }
}

/**
 * 悬浮控件入场标记: 地图就绪后置 true,
 * 让顶部胶囊 / 图例淡入并轻微下滑, 而不是"啪"地一下出现。
 */
@Local chromeIn: boolean = false;

/**
 * 相机变化时的标注刷新 (拖动地图时名字跟着走)。
 * 节流到约 60ms 一次: 太频繁会拖慢地图渲染, 太稀疏会看着"卡"。
 */

```

```

private onCameraChanged(): void {
  const now: number = Date.now();
  if (now - this.lastCameraTick < 60) {
    return;
  }
  this.lastCameraTick = now;
  this.updateCityLabels();
}

aboutToAppear(): void {
  this.safeTop = WindowService.safeTop;
  this.safeBottom = WindowService.safeBottom;
  this.mapOptions = {
    position: {
      target: { latitude: MapDefaults.centerLat, longitude: MapDefaults.centerLng },
      zoom: MapDefaults.zoom
    }
  };
  this.callback = (err: BusinessError | undefined, controller: map.MapComponentController | undefined) => {
    if (err !== undefined && err !== null) {
      this.hint = `地图加载失败 (错误码 ${err.code})。请确认已在 AGC 开通"地图服务"并重新申请调试 Profile。`;
      hilog.error(DOMAIN, TAG, 'map init failed: ${public}s', JSON.stringify(err));
    }
    return;
  }
  if (controller === undefined) {
    this.hint = '地图初始化失败';
    return;
  }
  this.controller = controller;
  this.setupMap(controller);
};
footprintStore.subscribe(this.listener);
this.loadLitColor();
this.startBlinking();
}

/**
 * 读取"点亮配色"并应用到地图。
 * 因为点位图标是预先烘好的 8 套 PNG，换色时要把所有已点亮 marker 的图标重下一遍。
 */
private async loadLitColor(): Promise<void> {
  try {
    const context = this.getUIContext().getHostContext() as common.UIAbilityContext;
    const index: number = await AchievementEngine.getLitColorIndex(context);
    if (index === this.litColorIndex) {
      return;
    }
    this.litColorIndex = index;
    this.applyLitColor();
  } catch (err) {

```

```
this.litColorIndex = 0;
}
}

/** 把所有已点亮城市的图标换成当前配色, 并刷新城市名颜色 */
private applyLitColor(): void {
  const icon: Resource = this.litIcon(this.litColorIndex);
  this.markers.forEach((marker: map.Marker, code: string) => {
    if (footprintStore.isVisited(code)) {
      marker.setIcon(icon);
    }
  });
  this.updateCityLabels();
}

/** 配色编号 → 点位图标资源 (预烘的 8 套 PNG) */
private litIcon(index: number): Resource {
  if (index === 1) {
    return $r('app.media.marker_lit_1');
  }
  if (index === 2) {
    return $r('app.media.marker_lit_2');
  }
  if (index === 3) {
    return $r('app.media.marker_lit_3');
  }
  if (index === 4) {
    return $r('app.media.marker_lit_4');
  }
  if (index === 5) {
    return $r('app.media.marker_lit_5');
  }
  if (index === 6) {
    return $r('app.media.marker_lit_6');
  }
  if (index === 7) {
    return $r('app.media.marker_lit_7');
  }
  return $r('app.media.marker_lit_0');
}

aboutToDisappear(): void {
  footprintStore.unsubscribe(this.listener);
  this.stopBlinking();
}

/**
 * 启动"已点亮点位闪烁"。
 *
 * 做法: 每 80ms 按 sine 曲线给每个已点亮 marker 下发一次透明度,
```

* 周期和相位由城市代码派生 — 每座城市都不一样，所以永远不会同步闪，

* 远远一看就是一片“呼吸着的金点”，去过哪里一目了然。

*/

```
private startBlinking(): void {
  if (this.blinkTimer !== -1) {
    return;
  }
  this.blinkTimer = setInterval(() => {
    this.tickBlink();
  }, MapDefaults.blinkIntervalMs);
}
```

```
private stopBlinking(): void {
  if (this.blinkTimer === -1) {
    return;
  }
  clearInterval(this.blinkTimer);
  this.blinkTimer = -1;
}
```

```
private tickBlink(): void {
  if (this.markers.size === 0) {
    return;
  }
  const now: number = Date.now();
  const span: number = MapDefaults.blinkBrightAlpha - MapDefaults.blinkDimAlpha;
  this.markers.forEach((marker: map.Marker, code: string) => {
    if (!footprintStore.isVisited(code)) {
      return;
    }
    const params: number[] = this.blinkParam(code);
    // sine 相位: 0 → 暗, 0.5 → 最亮
    const k: number = 0.5 + 0.5 * Math.sin(2 * Math.PI * (now / params[0] + params[1]));
    const alpha: number = MapDefaults.blinkDimAlpha + span * k;
    this.markerAlpha.set(code, alpha);
    marker.setAlpha(alpha);
  });
}
```

/** 由城市代码派生出 [周期ms, 相位0~1], 同一座城市每次都一样 */

```
private blinkParam(code: string): number[] {
  const cached: number[] | undefined = this.blinkParams.get(code);
  if (cached !== undefined) {
    return cached;
  }
  let h1: number = 17;
  let h2: number = 31;
  for (let i = 0; i < code.length; i++) {
    const c: number = code.charCodeAt(i);
    h1 = (h1 * 31 + c) % 100000;
  }
}
```

```

h2 = (h2 * 37 + c) % 100000;
}
const period: number = MapDefaults.blinkMinPeriodMs + (h1 % MapDefaults.blinkPeriodRangeMs);
const phase: number = (h2 % 1000) / 1000;
const params: number[] = [period, phase];
this.blinkParams.set(code, params);
return params;
}

/**
 * 地图初始化顺序很重要:
 * setCustomMapStyle 会清空地图上的覆盖物, 所以必须"先改样式、再画点位",
 * 否则刚加好的 370 个 marker 会被样式清掉。
 */
private async setupMap(controller: map.MapComponentController): Promise<void> {
  // 关键顺序: 先应用自定义样式 (会清空地图覆盖物), 再画点位
  await this.applyCustomStyle(controller);
  this.bindMapEvents(controller);
  await this.renderAllMarkers();
  this.updateCityLabels();
  // 点位就绪即视为地图可用, 清掉"加载中"提示 (mapLoad 事件在改样式后可能不再触发)
  this.mapReady = true;
  this.hint = '';
  // 让悬浮控件跟着淡入 (延迟一帧, 保证初值先渲染出来)
  setTimeout(() => {
    this.chromeIn = true;
  }, 60);
}

/** 应用 Petal Maps Studio 发布的地图样式 */
private async applyCustomStyle(controller: map.MapComponentController): Promise<void> {
  const options: mapCommon.CustomMapStyleOptions = { styleId: MAP_STYLE_ID };
  try {
    await controller.setCustomMapStyle(options);
    hilog.info(DOMAIN, TAG, 'custom map style applied: %{public}s', MAP_STYLE_ID);
  } catch (err) {
    const error: BusinessError = err as BusinessError;
    hilog.error(DOMAIN, TAG, 'setCustomMapStyle failed: %{public}d %{public}s', error.code, error.message);
  }
}

private bindMapEvents(controller: map.MapComponentController): void {
  // 合规: 显示审图号 (仅在境内显示), 这是"在地图显著位置标明审图号"要求的标准做法
  try {
    controller.setApproveNumberEnabled(true);
  } catch (err) {
    hilog.warn(DOMAIN, TAG, 'setApproveNumberEnabled not available');
  }
}

// 关闭底图自带控件: 缩放按钮 / 指南针 / 比例尺 / 定位按钮。

```

```
}  
}  
  
/** PurchaseData 字符串 → 结果对象 (商品不匹配 / 已退款 / 解析失败返回 undefined) */  
private static toOutcome(purchaseData: string): IapOutcome | undefined {  
  try {  
    const order: IapOrderPayload = JwsUtil.toOrderPayload(purchaseData);  
    if (order.productId !== PRODUCT_LIFETIME || !JwsUtil.isValidOrder(order)) {  
      return undefined;  
    }  
    return {  
      success: true,  
      canceled: false,  
      orderId: order.purchaseOrderId,  
      purchaseToken: order.purchaseToken,  
      productType: Number(order.productType),  
      sandbox: order.environment === 'SANDBOX',  
      message: ''  
    };  
  } catch (err) {  
    hilog.warn(DOMAIN, TAG, 'order decode failed');  
    return undefined;  
  }  
}  
  
private static failure(message: string): IapOutcome {  
  return {  
    success: false, canceled: false, orderId: '', purchaseToken: '',  
    productType: 0, sandbox: false, message: message  
  };  
}  
  
/** 错误码 → 用户能看懂的文案 */  
private static errorText(code: number): string {  
  if (code === iap.IAPErrorCode.USER_CANCELED) {  
    return '已取消购买';  
  }  
  if (code === iap.IAPErrorCode.SYSTEM_ERROR) {  
    return '系统繁忙, 请稍后重试';  
  }  
  if (code === iap.IAPErrorCode.APP_NOT_AUTHORIZED) {  
    return '应用内购买服务尚未生效, 请稍后重试';  
  }  
  if (code === iap.IAPErrorCode.INVALID_PRODUCT) {  
    return '商品不存在或已下架';  
  }  
  if (code === iap.IAPErrorCode.FREQUENT_CALLS) {  
    return '操作太频繁, 请稍后再试';  
  }  
  if (code === iap.IAPErrorCode.NETWORK_ERROR) {
```

```

return '网络异常, 请检查网络后重试';
}
if (code === iap.IAPErrorCode.PRODUCT_OWNED) {
return '该账号已经购买过, 正在为你恢复权益';
}
if (code === iap.IAPErrorCode.ACCOUNT_NOT_LOGGED_IN) {
return '请先在「设置 → 华为账号」登录后再开通';
}
if (code === iap.IAPErrorCode.PRODUCT_TERRITORY_NOT_SUPPORTED) {
return '当前华为账号的服务地暂不支持应用内购买';
}
if (code === 801) {
return '当前设备不支持应用内购买';
}
return '购买失败, 请稍后重试';
}
}

// ===== 文件: service/LocationService.ets =====
import { geoLocationManager } from '@kit.LocationKit';
import { abilityAccessCtrl, common, Permissions } from '@kit.AbilityKit';
import { hilog } from '@kit.PerformanceAnalysisKit';
import { CityItem } from '../model/CityDict';
import { CityRepository } from '../repository/CityRepository';

const DOMAIN: number = 0x0000;
const TAG: string = 'LocationService';

/**
 * 定位服务: 只取一次当前位置, 在本机换算成"最近的市州级城市"。
 * 隐私约定: 原始经纬度不落库、不上传。
 */
export class LocationService {
static async requestPermission(context: common.UIAbilityContext): Promise<boolean> {
const permissions: Permissions[] = [
'ohos.permission.APPROXIMATELY_LOCATION'
];
try {
const manager: abilityAccessCtrl.AtManager = abilityAccessCtrl.createAtManager();
const result = await manager.requestPermissionsFromUser(context, permissions);
const granted: boolean = result.authResults.length > 0 && result.authResults[0] === 0;
hilog.info(DOMAIN, TAG, 'permission granted: %{public}s', granted ? 'yes' : 'no');
return granted;
} catch (err) {
hilog.error(DOMAIN, TAG, 'request permission failed');
return false;
}
}

/** 返回"当前所在城市" (按最近城市中心点判定) */

```

```

static async detectCurrentCity(): Promise<CityItem | undefined> {
  try {
    const request: geoLocationManager.CurrentLocationRequest = {
      priority: geoLocationManager.LocationRequestPriority.FIRST_FIX,
      scenario: geoLocationManager.LocationRequestScenario.UNSET,
      timeoutMs: 5000
    };
    const location: geoLocationManager.Location = await geoLocationManager.getCurrentLocation(request);
    const city: CityItem | undefined = CityRepository.nearest(location.latitude, location.longitude);
    if (city !== undefined) {
      const distance: number = CityRepository.distanceKm(location.latitude, location.longitude, city.lat, city.lng);
      hilog.info(DOMAIN, TAG, 'nearest city %{public}s distance %{public}f km', city.name, distance);
    }
    return city;
  } catch (err) {
    hilog.error(DOMAIN, TAG, 'get current location failed');
    return undefined;
  }
}

// ===== 文件: service/MembershipService.ets =====
import { preferences } from '@kit.ArkData';
import { common } from '@kit.AbilityKit';
import { hilog } from '@kit.PerformanceAnalysisKit';

const DOMAIN: number = 0x0000;
const TAG: string = 'MembershipService';
const STORE_NAME: string = 'membership_store';
const KEY_MEMBER: string = 'is_member';
const KEY_FREE_CITIES: string = 'free_cities';
const KEY_LEGACY_CITIES: string = 'legacy_cities';
const KEY_ORDER_ID: string = 'order_id';
const KEY_SANDBOX: string = 'sandbox';
const KEY_GRANT_TIME: string = 'grant_time';
const KEY_MIGRATED: string = 'quota_migrated';

/** 免费体验额度: 终身可免费点亮 5 座城市 (第 6 座起需要终身会员) */
export const FREE_QUOTA: number = 5;

/**
 * 给界面观察的会员状态。
 * 状态管理 V2 的 @ObservedV2 + @Trace: 字段变化会自动刷新读到它的界面,
 * 所以「地图上的免费名额」「我的页的会员卡片」「购买面板」都读这一个对象,
 * 购买成功后不需要各页面手动刷新。
 */
@ObservedV2
export class MembershipState {
  @Trace member: boolean = false;
  @Trace freeUsed: number = 0;

```

```

@Trace orderShort: string = '';
@Trace sandbox: boolean = false;
@Trace grantTime: number = 0;
}

export const membershipState: MembershipState = new MembershipState();

/**
 * 会员与免费额度。
 *
 * 产品规则（对应《设计前指导书》付费章节）：
 * 1. 免费额度是**终身累计**的 5 座城市，不是每月刷新；
 * 2. 记录的是"用掉额度的城市编码集合"，不是简单计数器——
 * 这样"取消点亮再重新点亮同一座城市"不会重复扣额度，
 * 而"点亮→取消→换一座城市点亮"也不能绕过限制；
 * 3. 已经点亮的城市永久保留：老用户（升级前就点亮了 20 座）不会被回滚，
 * 只是"再点亮新城市"时才需要开通会员。升级时（ensureMigration）会把这批
 * 存量城市全部记为 legacy：其中前 5 座算作"已用免费额度"，其余的城市
 * 虽然计入免费额度之外，但**永远允许重新点亮**（取消后还能再点回来）。
 */
export class MembershipService {
  private static store: preferences.Preferences | undefined = undefined;
  private static member: boolean = false;
  private static freeCities: string[] = [];
  /**
   * 升级到付费版之前就已经点亮的城市（存量数据）。
   * 这些城市永远可以重新点亮、不消耗免费额度：
   * 否则老用户一旦手滑取消点亮某座城市，就再也不能点亮它——那是很糟糕的体验。
   */
  private static legacyCities: string[] = [];
  private static orderId: string = '';
  private static sandbox: boolean = false;
  private static grantTime: number = 0;

  /** 启动时调用一次 */
  static async init(context: common.UIAbilityContext): Promise<void> {
    if (MembershipService.store !== undefined) {
      return;
    }
    try {
      const options: preferences.Options = { name: STORE_NAME };
      const store: preferences.Preferences = await preferences.getPreferences(context, options);
      MembershipService.store = store;
      MembershipService.member = store.getSync(KEY_MEMBER, false) as boolean;
      MembershipService.orderId = store.getSync(KEY_ORDER_ID, '') as string;
      MembershipService.sandbox = store.getSync(KEY_SANDBOX, false) as boolean;
      MembershipService.grantTime = store.getSync(KEY_GRANT_TIME, 0) as number;
      const raw: string = store.getSync(KEY_FREE_CITIES, '') as string;
      MembershipService.freeCities = MembershipService.parseList(raw);
      const legacy: string = store.getSync(KEY_LEGACY_CITIES, '') as string;

```

```
MembershipService.legacyCities = MembershipService.parseList(legacy);
MembershipService.publish();
hilog.info(DOMAIN, TAG, 'membership restored: member=%{public}d freeUsed=%{public}d',
MembershipService.member ? 1 : 0, MembershipService.freeCities.length);
} catch (err) {
hilog.warn(DOMAIN, TAG, 'init failed: %{public}s', JSON.stringify(err));
}
}

static isMember(): boolean {
return MembershipService.member;
}

static isSandbox(): boolean {
return MembershipService.sandbox;
}

static grantedAt(): number {
return MembershipService.grantTime;
}

/** 已用掉的免费额度 (0~5) */
static freeUsedCount(): number {
return MembershipService.freeCities.length;
}

/** 剩余免费额度 */
static freeLeft(): number {
const left: number = FREE_QUOTA - MembershipService.freeCities.length;
return left > 0 ? left : 0;
}

/**
* 点亮这座城市是否会消耗一个免费名额。
* 会员、已经用过额度的城市、以及升级前就存在的老数据都不消耗。
* 批量点亮时用它做"实时扣减", 否则同一批里前几座刚用掉的名额不会被算进去。
*/
static usesFreeSlot(code: string): boolean {
if (MembershipService.member) {
return false;
}
if (MembershipService.freeCities.includes(code)) {
return false;
}
if (MembershipService.legacyCities.includes(code)) {
return false;
}
return true;
}
```

```
/**
 * 这座城市的"点亮"是否需要先开通会员。
 * 已经点亮的城市（含老用户存量数据）永远允许重新点亮，不消耗新额度。
 */
static canLight(code: string): boolean {
  if (MembershipService.member) {
    return true;
  }
  if (MembershipService.freeCities.includes(code)) {
    return true;
  }
  if (MembershipService.legacyCities.includes(code)) {
    return true;
  }
  return MembershipService.freeCities.length < FREE_QUOTA;
}

/** 用掉一个免费额度（会员 / 已记录过的城市不重复计） */
static async consumeFreeSlot(code: string): Promise<void> {
  if (MembershipService.member || MembershipService.freeCities.includes(code)) {
    return;
  }
  if (MembershipService.freeCities.length >= FREE_QUOTA) {
    return;
  }
  const next: string[] = MembershipService.freeCities.slice();
  next.push(code);
  MembershipService.freeCities = next;
  await MembershipService.persist(KEY_FREE_CITIES, JSON.stringify(next));
  MembershipService.publish();
}

/**
 * 老用户迁移：升级到付费版之前已经点亮的城市不回滚，
 * 只把它们中的前 5 座记为"已用免费额度"，之后点亮新城市就需要会员。
 * 只执行一次。
 */
static async ensureMigration(visitedCodes: string[]): Promise<void> {
  const store: preferences.Preferences | undefined = MembershipService.store;
  if (store === undefined) {
    return;
  }
  try {
    const migrated: boolean = store.getSync(KEY_MIGRATED, false) as boolean;
    if (migrated) {
      return;
    }
    const next: string[] = MembershipService.freeCities.slice();
    const legacy: string[] = [];
    for (const code of visitedCodes) {
```

```

if (!legacy.includes(code)) {
  legacy.push(code);
}
if (next.length < FREE_QUOTA && !next.includes(code)) {
  next.push(code);
}
}
MembershipService.freeCities = next;
MembershipService.legacyCities = legacy;
await store.put(KEY_MIGRATED, true);
await store.put(KEY_FREE_CITIES, JSON.stringify(next));
await store.put(KEY_LEGACY_CITIES, JSON.stringify(legacy));
await store.flush();
MembershipService.publish();
hilog.info(DOMAIN, TAG, 'quota migrated, freeUsed=%{public}d', next.length);
} catch (err) {
  hilog.warn(DOMAIN, TAG, 'migration failed: %{public}s', JSON.stringify(err));
}
}

/** 购买成功 / 恢复购买成功: 记为终身会员 */
static async grant(orderId: string, sandbox: boolean): Promise<void> {
  MembershipService.member = true;
  MembershipService.orderId = orderId;
  MembershipService.sandbox = sandbox;
  MembershipService.grantTime = Date.now();
  const store: preferences.Preferences | undefined = MembershipService.store;
  if (store === undefined) {
    return;
  }
  await store.put(KEY_MEMBER, true);
  await store.put(KEY_ORDER_ID, orderId);
  await store.put(KEY_SANDBOX, sandbox);
  await store.put(KEY_GRANT_TIME, MembershipService.grantTime);
  await store.flush();
  MembershipService.publish();
  hilog.info(DOMAIN, TAG, 'membership granted, sandbox=%{public}d', sandbox ? 1 : 0);
}

/** 订单号后 6 位, 用于界面展示 (不展示完整订单号) */
static orderShortId(): string {
  const id: string = MembershipService.orderId;
  if (id.length <= 6) {
    return id;
  }
  return `...${id.substring(id.length - 6)}`;
}

/**
 * 仅开发调试使用: 把本机会员状态和免费额度清空,

```

* 方便反复验证"第 6 座城市触发付费"的完整流程。

* 入口只出现在 debug 构建里 (见 SettingsPage 的 BuildProfile.DEBUG 判断)。

```

*/
static async resetForDebug(): Promise<void> {
  MembershipService.member = false;
  MembershipService.freeCities = [];
  MembershipService.legacyCities = [];
  MembershipService.orderId = '';
  MembershipService.sandbox = false;
  MembershipService.grantTime = 0;
  const store: preferences.Preferences | undefined = MembershipService.store;
  if (store === undefined) {
    return;
  }
  await store.put(KEY_MEMBER, false);
  await store.put(KEY_FREE_CITIES, '');
  await store.put(KEY_LEGACY_CITIES, '');
  await store.put(KEY_ORDER_ID, '');
  await store.put(KEY_SANDBOX, false);
  await store.put(KEY_GRANT_TIME, 0);
  await store.flush();
  MembershipService.publish();
  hilog.info(DOMAIN, TAG, 'membership reset (debug only)');
}

/** 把内存状态同步到可观察对象 (界面据此刷新) */
private static publish(): void {
  membershipState.member = MembershipService.member;
  membershipState.freeUsed = MembershipService.freeCities.length;
  membershipState.sandbox = MembershipService.sandbox;
  membershipState.orderShort = MembershipService.orderShortId();
  membershipState.grantTime = MembershipService.grantTime;
}

private static async persist(key: string, value: string): Promise<void> {
  const store: preferences.Preferences | undefined = MembershipService.store;
  if (store === undefined) {
    return;
  }
  await store.put(key, value);
  await store.flush();
}

private static parseList(raw: string): string[] {
  if (raw.length === 0) {
    return [];
  }
  try {
    const parsed: string[] = JSON.parse(raw) as string[];
    return parsed === null ? [] : parsed;
  }

```

```

} catch (err) {
return [];
}
}
}

// ===== 文件: repository/CityRepository.ets =====
import { common } from '@kit.AbilityKit';
import { util } from '@kit.ArkTS';
import { hilog } from '@kit.PerformanceAnalysisKit';
import { CityDictFile, CityItem, CountryItem, ProvinceItem } from '../model/CityDict';

const DOMAIN: number = 0x0000;
const TAG: string = 'CityRepository';
const RAW_FILE: string = 'city_dict_v1.json';

/**
 * 城市字典仓储: 字典随包发布 (rawfile) , 只读, 不打数据库、不联网。
 * 数据来源: 公开行政区划边界数据 (GCJ-02) + 民政部行政区划代码。
 */
export class CityRepository {
private static cities: CityItem[] = [];
private static byCode: Map<string, CityItem> = new Map<string, CityItem>();
private static provinceList: string[] = [];
private static provinceLabelItems: ProvinceItem[] = [];
private static countryLabelItems: CountryItem[] = [];
private static version: string = '';
private static generatedAt: string = '';

static async load(context: common.UIAbilityContext): Promise<void> {
if (CityRepository.cities.length > 0) {
return;
}
const bytes: Uint8Array = await context.resourceManager.getRawFileContent(RAW_FILE);
const text: string = util.TextDecoder.create('utf-8').decodeToString(bytes);
const file: CityDictFile = JSON.parse(text) as CityDictFile;
CityRepository.cities = file.cities;
CityRepository.version = file.version;
CityRepository.generatedAt = file.generatedAt;

const index: Map<string, CityItem> = new Map<string, CityItem>();
const provinces: string[] = [];
for (const city of file.cities) {
index.set(city.code, city);
if (city.parent.length > 0 && !provinces.includes(city.parent)) {
provinces.push(city.parent);
}
}
CityRepository.byCode = index;
CityRepository.provinceList = provinces;

```

```
CityRepository.provinceLabelItems = file.provinces === undefined ? [] : file.provinces;
CityRepository.countryLabelItems = file.countries === undefined ? [] : file.countries;
hilog.info(DOMAIN, TAG, 'city dict loaded: %{public}d units, %{public}d provinces', file.cities.length,
provinces.length);
}

static all(): CityItem[] {
return CityRepository.cities;
}

static get(code: string): CityItem | undefined {
return CityRepository.byCode.get(code);
}

static total(): number {
return CityRepository.cities.length;
}

static provinces(): string[] {
return CityRepository.provinceList;
}

/** 省级行政区列表（含标签坐标），用于在地图上绘制省份名 */
static provinceItems(): ProvinceItem[] {
return CityRepository.provinceLabelItems;
}

/** 国家 / 地区名列表（自绘，替代底图自带的国名） */
static countryItems(): CountryItem[] {
return CityRepository.countryLabelItems;
}

static dictVersion(): string {
return CityRepository.version;
}

static dictGeneratedAt(): string {
return CityRepository.generatedAt;
}

/** 模糊搜索：支持全称、简称、拼音首字母（暂用名称匹配） */
static search(keyword: string): CityItem[] {
const key: string = keyword.trim();
if (key.length === 0) {
return CityRepository.cities;
}
const result: CityItem[] = [];
for (const city of CityRepository.cities) {
if (city.name.includes(key) || city.short.includes(key) || city.parent.includes(key)) {
result.push(city);
}
```

```

}
}
return result;
}

/** 找到距离给定坐标最近的城市的（用于：点按地图 → 猜你想点亮哪座城市） */
static nearest(lat: number, lng: number): CityItem | undefined {
  let best: CityItem | undefined = undefined;
  let bestDistance: number = Number.MAX_VALUE;
  for (const city of CityRepository.cities) {
    const d: number = CityRepository.distanceKm(lat, lng, city.lat, city.lng);
    if (d < bestDistance) {
      bestDistance = d;
      best = city;
    }
  }
  return best;
}

/** 两点球面距离（公里） */
static distanceKm(lat1: number, lng1: number, lat2: number, lng2: number): number {
  const rad: number = Math.PI / 180;
  const dLat: number = (lat2 - lat1) * rad;
  const dLng: number = (lng2 - lng1) * rad;
  const a: number = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(lat1 * rad) * Math.cos(lat2 * rad) * Math.sin(dLng / 2) * Math.sin(dLng / 2);
  return 6371 * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
}

// ===== 文件: repository/DatabaseHelper.ets =====
import { relationalStore } from '@kit.ArkData';
import { common } from '@kit.AbilityKit';
import { BusinessError } from '@kit.BasicServicesKit';
import { hilog } from '@kit.PerformanceAnalysisKit';

const DOMAIN: number = 0x0000;
const TAG: string = 'DatabaseHelper';

const DDL_VISIT: string = 'CREATE TABLE IF NOT EXISTS visit_record (' +
  'id TEXT PRIMARY KEY, ' +
  'adcode TEXT NOT NULL, ' +
  'visit_date INTEGER, ' +
  'leave_date INTEGER, ' +
  'days INTEGER, ' +
  'tag TEXT, ' +
  'note TEXT, ' +
  'companions TEXT, ' +
  'created_at INTEGER NOT NULL, ' +
  'updated_at INTEGER NOT NULL, ' +

```

```
'deleted INTEGER DEFAULT 0)';

const DDL_INDEX_ADcode: string = 'CREATE INDEX IF NOT EXISTS idx_visit_adcode ON visit_record(adcode)';
const DDL_INDEX_DATE: string = 'CREATE INDEX IF NOT EXISTS idx_visit_date ON visit_record(visit_date DESC)';
const DDL_INDEX_DIRTY: string = 'CREATE INDEX IF NOT EXISTS idx_visit_dirty ON visit_record(updated_at)';

/**
 * 关系型数据库初始化与迁移。
 * 说明：全部数据仅保存在本机沙箱；首版不做云端同步。
 */
export class DatabaseHelper {
  private static store: relationalStore.RdbStore | undefined = undefined;

  static async init(context: common.UIAbilityContext): Promise<void> {
    if (DatabaseHelper.store !== undefined) {
      return;
    }
    try {
      const config: relationalStore.StoreConfig = {
        name: 'footprint.db',
        securityLevel: relationalStore.SecurityLevel.S1
      };
      const store: relationalStore.RdbStore = await relationalStore.getRdbStore(context, config);
      await store.executeSql(DDL_VISIT);
      await store.executeSql(DDL_INDEX_ADcode);
      await store.executeSql(DDL_INDEX_DATE);
      await store.executeSql(DDL_INDEX_DIRTY);
      DatabaseHelper.store = store;
      hilog.info(DOMAIN, TAG, 'database ready');
    } catch (err) {
      const error: BusinessError = err as BusinessError;
      hilog.error(DOMAIN, TAG, 'init database failed: %{public}s', error.message);
      throw new Error(`数据库初始化失败: ${error.message}`);
    }
  }

  static requireStore(): relationalStore.RdbStore {
    const store: relationalStore.RdbStore | undefined = DatabaseHelper.store;
    if (store === undefined) {
      throw new Error('database is not initialized');
    }
    return store;
  }
}

// ===== 文件: repository/VisitRepository.ets =====
import { relationalStore } from '@kit.ArkData';
import { DatabaseHelper } from '../DatabaseHelper';
import { CityVisitStat, VisitRecord } from '../model/VisitRecord';
```

```
const TABLE: string = 'visit_record';

/** 到访记录读写 */
export class VisitRepository {
  /** 新增一条到访记录 (点亮城市) */
  static async addVisit(adcode: string, visitDate: number, tag: string, note: string, days: number): Promise<void> {
    const store: relationalStore.RdbStore = DatabaseHelper.requireStore();
    const now: number = Date.now();
    const values: relationalStore.ValuesBucket = {
      'id': `${adcode}_${now}_${Math.floor(Math.random() * 10000)}`,
      'adcode': adcode,
      'visit_date': visitDate,
      'leave_date': 0,
      'days': days,
      'tag': tag,
      'note': note,
      'companions': '',
      'created_at': now,
      'updated_at': now,
      'deleted': 0
    };
    await store.insert(TABLE, values);
  }

  /** 批量点亮: 单事务, 保证原子性 */
  static async addVisitsBatch(adcodes: string[]): Promise<void> {
    const store: relationalStore.RdbStore = DatabaseHelper.requireStore();
    const now: number = Date.now();
    store.beginTransaction();
    try {
      for (const adcode of adcodes) {
        const values: relationalStore.ValuesBucket = {
          'id': `${adcode}_${now}_${Math.floor(Math.random() * 10000)}`,
          'adcode': adcode,
          'visit_date': now,
          'leave_date': 0,
          'days': 0,
          'tag': 'other',
          'note': '',
          'companions': '',
          'created_at': now,
          'updated_at': now,
          'deleted': 0
        };
        await store.insert(TABLE, values);
      }
      store.commit();
    } catch (err) {
      store.rollback();
      throw err as Error;
    }
  }
}
```

```

}
}

```

```

/** 取消点亮：软删除该城市全部到访记录 */
static async removeCity(adcode: string): Promise<void> {
  const store: relationalStore.RdbStore = DatabaseHelper.requireStore();
  const predicates: relationalStore.RdbPredicates = new relationalStore.RdbPredicates(TABLE);
  predicates.equalTo('adcode', adcode);
  const values: relationalStore.ValuesBucket = {
    'deleted': 1,
    'updated_at': Date.now()
  };
  await store.update(values, predicates);
}

```

```

/**
 * 清除本机全部足迹记录（「我的 → 清除本机足迹数据」）。
 *
 * 用软删除而不是 DELETE：与本仓库其它写操作保持一致，
 * 云同步的合并逻辑靠 deleted 标记判断"这条记录被删过"，
 * 直接物理删除会让下次全量同步把旧记录当成新增又拉回来。
 */
static async clearAll(): Promise<void> {
  const store: relationalStore.RdbStore = DatabaseHelper.requireStore();
  const predicates: relationalStore.RdbPredicates = new relationalStore.RdbPredicates(TABLE);
  predicates.notEqualTo('deleted', 1);
  const values: relationalStore.ValuesBucket = {
    'deleted': 1,
    'updated_at': Date.now()
  };
  await store.update(values, predicates);
}

```

```

/** 读取全部已点亮的城市编码 */
static async loadVisitedCodes(): Promise<string[]> {
  const store: relationalStore.RdbStore = DatabaseHelper.requireStore();
  const predicates: relationalStore.RdbPredicates = new relationalStore.RdbPredicates(TABLE);
  predicates.equalTo('deleted', 0);
  const resultSet: relationalStore.ResultSet = await store.query(predicates, ['adcode']);
  const codes: string[] = [];
  const column: number = resultSet.getColumnIndex('adcode');
  while (resultSet.goToNextRow()) {
    const code: string = resultSet.getString(column);
    if (!codes.includes(code)) {
      codes.push(code);
    }
  }
  resultSet.close();
  return codes;
}

```

```
/** 读取每个城市的到访统计（次数、首次、最近） */
static async loadCityStats(): Promise<CityVisitStat[]> {
  const store: relationalStore.RdbStore = DatabaseHelper.requireStore();
  const predicates: relationalStore.RdbPredicates = new relationalStore.RdbPredicates(TABLE);
  predicates.equalTo('deleted', 0);
  const resultSet: relationalStore.ResultSet = await store.query(predicates, ['adcode', 'visit_date']);
  const index: Map<string, CityVisitStat> = new Map<string, CityVisitStat>();
  const colCode: number = resultSet.getColumnIndex('adcode');
  const colDate: number = resultSet.getColumnIndex('visit_date');
  while (resultSet.moveToNextRow()) {
    const code: string = resultSet.getString(colCode);
    const date: number = resultSet.getLong(colDate);
    const exist: CityVisitStat | undefined = index.get(code);
    if (exist === undefined) {
      index.set(code, { adcode: code, count: 1, firstVisit: date, lastVisit: date });
    } else {
      exist.count = exist.count + 1;
      exist.lastVisit = date > exist.lastVisit ? date : exist.lastVisit;
      exist.firstVisit = date < exist.firstVisit ? date : exist.firstVisit;
    }
  }
  resultSet.close();
  const stats: CityVisitStat[] = [];
  index.forEach((value: CityVisitStat) => {
    stats.push(value);
  });
  return stats;
}

/** 读取全部记录（含软删除），用于云同步 */
static async loadAllRecords(): Promise<VisitRecord[]> {
  const store: relationalStore.RdbStore = DatabaseHelper.requireStore();
  const predicates: relationalStore.RdbPredicates = new relationalStore.RdbPredicates(TABLE);
  const resultSet: relationalStore.ResultSet = await store.query(predicates,
    ['id', 'adcode', 'visit_date', 'leave_date', 'days', 'tag', 'note', 'created_at', 'updated_at', 'deleted']);
  const list: VisitRecord[] = [];
  const cId: number = resultSet.getColumnIndex('id');
  const cAdcode: number = resultSet.getColumnIndex('adcode');
  const cVisit: number = resultSet.getColumnIndex('visit_date');
  const cLeave: number = resultSet.getColumnIndex('leave_date');
  const cDays: number = resultSet.getColumnIndex('days');
  const cTag: number = resultSet.getColumnIndex('tag');
  const cNote: number = resultSet.getColumnIndex('note');
  const cCreated: number = resultSet.getColumnIndex('created_at');
  const cUpdated: number = resultSet.getColumnIndex('updated_at');
  const cDeleted: number = resultSet.getColumnIndex('deleted');
  while (resultSet.moveToNextRow()) {
    list.push({
      id: resultSet.getString(cId),
```

```

adcode: resultSet.getString(cAdcode),
visitDate: resultSet.getLong(cVisit),
leaveDate: resultSet.getLong(cLeave),
days: resultSet.getLong(cDays),
tag: resultSet.getString(cTag),
note: resultSet.getString(cNote),
createdAt: resultSet.getLong(cCreated),
updatedAt: resultSet.getLong(cUpdated),
deleted: resultSet.getLong(cDeleted)
});
}
resultSet.close();
return list;
}

/**
 * 云同步落库：按 id 覆盖写（本地更新的记录不被旧数据覆盖）。
 * 返回真正写入/更新的条数。
 */
static async mergeFromCloud(records: VisitRecord[]): Promise<number> {
  if (records.length === 0) {
    return 0;
  }
  const store: relationalStore.RdbStore = DatabaseHelper.requireStore();
  let written: number = 0;
  store.beginTransaction();
  try {
    for (const record of records) {
      // 先看本地有没有这条、以及谁的 updatedAt 更新
      const probe: relationalStore.RdbPredicates = new relationalStore.RdbPredicates(TABLE);
      probe.equalTo('id', record.id);
      const rs: relationalStore.ResultSet = await store.query(probe, ['updated_at']);
      let localUpdated: number = -1;
      if (rs.goToNextRow()) {
        localUpdated = rs.getLong(rs.getColumnIndex('updated_at'));
      }
      rs.close();
      if (localUpdated >= record.updatedAt) {
        continue; // 本地更新，保留本地
      }
      const values: relationalStore.ValuesBucket = {
        'id': record.id,
        'adcode': record.adcode,
        'visit_date': record.visitDate,
        'leave_date': record.leaveDate,
        'days': record.days,
        'tag': record.tag,
        'note': record.note,
        'companions': '',
        'created_at': record.createdAt,

```

```
'updated_at': record.updatedAt,
'deleted': record.deleted
};
await store.insert(TABLE, values,
relationalStore.ConflictResolution.ON_CONFLICT_REPLACE);
written = written + 1;
}
store.commit();
} catch (err) {
store.rollback();
throw err as Error;
}
return written;
}
}
```

```
// ===== 文件: model/CityDict.ets =====
```

```
/** 点亮单元层级 */
```

```
export enum CityLevel {
```

```
/** 直辖市 */
```

```
MUNICIPALITY = 'municipality',
```

```
/** 地级市 / 自治州 / 地区 / 盟 / 省直辖县级 */
```

```
CITY = 'city',
```

```
/** 特别行政区 */
```

```
SAR = 'sar',
```

```
/** 省级整体 (台湾省) */
```

```
PROVINCE = 'province'
```

```
}
```

```
/** 城市字典条目 (一个可点亮的市州级单元) */
```

```
export interface CityItem {
```

```
/** 行政区划代码, 唯一主键 */
```

```
code: string;
```

```
/** 全称, 如"杭州市" */
```

```
name: string;
```

```
/** 简称, 如"杭州" */
```

```
short: string;
```

```
/** 层级, 取值见 CityLevel */
```

```
level: string;
```

```
/** 所属省级行政区名称 */
```

```
parent: string;
```

```
/** 所属省级行政区代码 */
```

```
parentCode: string;
```

```
/** 中心点经度 (GCJ-02) */
```

```
lng: number;
```

```
/** 中心点纬度 (GCJ-02) */
```

```
lat: number;
```

```
}
```

```
/** 省级行政区 (用于地图上的省份名标注) */
```

```

export interface ProvinceItem {
  /** 省级行政区划代码 */
  code: string;
  /** 全称 */
  name: string;
  /** 简称: 内蒙古自治区 → 内蒙古 */
  short: string;
  /** 标签位置 (几何重心, GCJ-02) */
  lng: number;
  lat: number;
  /** 该省行政中心的行政区划代码 (省会) */
  capitalCode: string;
}

/**
 * 国家 / 地区名 (自绘)。
 * 底图自带的国名会被画在中国正中央、压住河北/山西/宁夏几个省名,
 * 且位置不可控; 因此改为在样式里把「国家」整类隐藏, 由 App 自己绘制。
 */
export interface CountryItem {
  /** 显示名, 如"蒙古" */
  name: string;
  /** 标签锚点经度 (GCJ-02) */
  lng: number;
  lat: number;
}

/** 城市字典文件结构 */
export interface CityDictFile {
  version: string;
  generatedAt: string;
  source: string;
  total: number;
  countries?: CountryItem[];
  provinces: ProvinceItem[];
  cities: CityItem[];
}

// ===== 文件: model/IapModels.ets =====
import { util } from '@kit.ArkTS';

/**
 * IAP Kit 数据模型 (字段与官方《IAP Kit 数据模型》一致)。
 *
 * 数据链路:
 * createPurchase / queryPurchases
 * → PurchaseData (JSON 字符串, 含 jwsPurchaseOrder)
 * → 解码 JWS 载荷得到 PurchaseOrderPayload (真正可用的订单信息)
 *
 * 官方文档:

```

```

* · 接入购买: harmonyos-guides/iap-integrate-purchase
* · 权益发放: harmonyos-guides/iap-delivering-products
* · 数据模型: harmonyos-references/iap-data-model
*/

/** createPurchase / queryPurchases 返回的 PurchaseData */
export interface IapPurchaseData {
/** 商品类型: 0 消耗型 / 1 非消耗型 / 2 自动续期 / 3 非续期订阅 */
type: number;
/** 订单信息 (JWS 格式), 本 App 只用到这一个字段 */
jwsPurchaseOrder?: string;
jwsSubscriptionStatus?: string;
}

/** 解码 jwsPurchaseOrder 后的订单信息 */
export interface IapOrderPayload {
/** NORMAL 生产环境 / SANDBOX 沙盒环境 */
environment: string;
/** 购买订单号 (finishPurchase 需要) */
purchaseOrderId: string;
/** 购买令牌 (finishPurchase 需要) */
purchaseToken: string;
applicationId: string;
productId: string;
/** 商品类型, 注意官方这里是**字符串**: "1" 表示非消耗型 */
productType: string;
quantity?: number;
purchaseTime: number;
/** "1" 已发货 / "2" 未发货 */
finishStatus?: string;
needFinish?: boolean;
/** 实付价格, 单位: 分 */
price: number;
/** 币种, 如 CNY */
currency: string;
developerPayload?: string;
/** 有值代表订单已撤销 (退款), 此时不应发货 */
purchaseOrderRevocationReasonCode?: string;
revocationTime?: number;
offerTypeCode?: string;
offerId?: string;
}

/**
* JWS 解码工具。
*
* 说明 (与官方口径保持一致):
* · 官方建议把 purchaseData 发给应用服务器做**验签**, 本 App 是单机应用
* (官方《权益发放》明确: 单机应用的服务器交互放在客户端完成),
* 所以这里只做**载荷解码**, 订单本身来自 IAP Kit 系统接口而非第三方输入,

```

```

* 配合启动时向华为侧复查已购列表，足够防“本地篡改会员标志”这一级别的风险；
* · 验签需要内置华为根证书链，属于服务端方案，本 App 暂不引入。
*/
export class JwsUtil {
  /** 取出 JWS 的载荷部分（第二段）并解成 JSON 字符串 */
  static decodePayload(jws: string): string {
    const parts: string[] = jws.split('.');
    if (parts.length < 2 || parts[1].length === 0) {
      throw new Error('非法订单数据');
    }
    return JwsUtil.base64UrlToString(parts[1]);
  }

  /**
   * Base64URL → 字符串。
   * JWS 用的是 base64url 变体（把 +/ 换成 -_，且去掉末尾 =），
   * 系统解码器只认标准 base64，所以要先还原字符并补齐 padding。
   */
  static base64UrlToString(input: string): string {
    let text: string = input.replace(/-/g, '+').replace(/_/g, '/');
    while (text.length % 4 !== 0) {
      text += '=';
    }
    const bytes: Uint8Array = new util.Base64Helper().decodeSync(text);
    return util.TextDecoder.create('utf-8').decodeToString(bytes);
  }

  /** 解析 PurchaseData 字符串 */
  static parsePurchaseData(purchaseData: string): IapPurchaseData {
    return JSON.parse(purchaseData) as IapPurchaseData;
  }

  /** PurchaseData → PurchaseOrderPayload */
  static toOrderPayload(purchaseData: string): IapOrderPayload {
    const data: IapPurchaseData = JwsUtil.parsePurchaseData(purchaseData);
    const jws: string | undefined = data.jwsPurchaseOrder;
    if (jws === undefined || jws.length === 0) {
      throw new Error('订单数据缺少 jwsPurchaseOrder');
    }
    return JSON.parse(JwsUtil.decodePayload(jws)) as IapOrderPayload;
  }

  /** 订单是否有效（未撤销，即没有退款） */
  static isValidOrder(order: IapOrderPayload): boolean {
    const code: string | undefined = order.purchaseOrderRevocationReasonCode;
    return code === undefined || code.length === 0;
  }
}

// ===== 文件: model/VisitRecord.ets =====

```

```
/** 到访标签 */
export enum VisitTag {
  BUSINESS = 'business',
  TRAVEL = 'travel',
  FAMILY = 'family',
  TRANSIT = 'transit',
  LIVE = 'live',
  OTHER = 'other'
}

/** 一条到访记录 */
export interface VisitRecord {
  id: string;
  adcode: string;
  visitDate: number;
  leaveDate: number;
  days: number;
  tag: string;
  note: string;
  createdAt: number;
  updatedAt: number;
  deleted: number;
}

/** 单个城市的到访汇总 */
export interface CityVisitStat {
  adcode: string;
  count: number;
  firstVisit: number;
  lastVisit: number;
}

/** 标签的中文名 */
export function tagLabel(tag: string): string {
  switch (tag) {
    case VisitTag.BUSINESS:
      return '出差';
    case VisitTag.TRAVEL:
      return '旅行';
    case VisitTag.FAMILY:
      return '探亲';
    case VisitTag.TRANSIT:
      return '路过';
    case VisitTag.LIVE:
      return '居住';
    default:
      return '其他';
  }
}
```

```
// ===== 文件: common/DeveloperInfo.ets =====
/**
 * 开发者信息（对外公示用，单一数据源）。
 *
 * 为什么单独抽一个文件：
 * 1. 隐私政策（应用内 + 网页版）、「我的 → 关于」、以及商户服务申请材料里的
 * 「隐私页 / 关于页截图」必须是**同一套信息**，散落各处迟早对不上；
 * 2. 华为商户服务审核明确要求：截图需展示"应用由xxx开发"，
 * 且应用名称要能对上，所以这几项在这里统一定义。
 */
export class DeveloperInfo {
  /** 应用名称（必须与 module.json5 的 label 一致） */
  static readonly APP_NAME: string = '走遍中国';
  /** 开发者署名：审核要求能看出"应用由谁开发" */
  static readonly DEVELOPER: string = '吕志方（个人开发者）';
  /**
   * 对外联系邮箱：出现在隐私政策与「关于」页，用于审核联系与用户投诉。
   * 必须换成真实可用的邮箱后再出商户服务材料截图。
   */
  static readonly CONTACT_EMAIL: string = 'lvzhifang2015@163.com';
  /** 版权署名 */
  static readonly COPYRIGHT: string = '© 2026 吕志方';
}

// ===== 文件: common/MaterialKit.ets =====
import uiMaterial from '@ohos.arkui.uiMaterial';
import { deviceInfo } from '@kit.BasicServicesKit';
import { hilog } from '@kit.PerformanceAnalysisKit';
import { Palette } from './Theme';

const DOMAIN: number = 0x0000;
const TAG: string = 'MaterialKit';

/**
 * 沉浸光感材质（HarmonyOS 7 / API 26 新增的 uiMaterial）。
 *
 * 一条踩过坑的硬约束（官方《沉浸光感常见问题》"组件不在沉浸光感生效范围")：
 * 除了弹窗/半模态/菜单这类浮层，以及 Slider/Toggle/Select 之外，
 * **其他组件（包括 Button）的 systemMaterial 只在 Navigation/NavDestination 标题栏
 * 或横向 Tabs 且 barPosition=BarPosition.End 的底部 TabBar 内生效**。


放错地方时系统只会打一条日志：`Material inactive: out of scope.`，界面上毫无变化。



所以本 App：



- 底部页签栏 — 走 Tabs.barFloatingStyle（本文件的 floatingTabBar()），材质真的会渲染；
- 地图上的悬浮胶囊/圆按钮 — 不走材质，直接用 backgroundBlurStyle 做真模糊。



（同时用 backgroundColor 和材质也不行：材质画在背板层，背景色画在内容层，会把材质盖掉。）



材质语义（对应鸿蒙 7 "沉浸光感"设计语言）：



- style — 材质厚度，越薄越通透，越厚越"磨砂"；
- materialColor — 材质层染色，必须是带透明度的颜色，全不透明会挡掉滤色；
- interactive — 按压形变反馈；

```

```

* · lightEffect — 光感交互反馈（手指交互时材质表面出现的光晕颜色）。
*
* 使用方式: EntryAbility / Index.aboutToAppear 调用 MaterialKit.init(),
* 页签栏样式里用 `MaterialKit.floatingTabBar()`.
*/
export class MaterialKit {
  /** 光感反馈色: 金色光晕, 贴合"点亮足迹"的产品语言 */
  private static readonly GLOW: string = '#7AFFD97D';
  /** 是否支持沉浸光感 (鸿蒙 7 / API 26+) */
  private static supported: boolean = false;
  /** 初始化失败的诊断信息 (用于界面上显示, 方便定位为什么不生效) */
  static lastError: string = 'not-initialized';
  /**
   * 诊断串: 把"系统是否真的会把材质渲染出来"的关键参数记下来,
   * 显示在「我的 → 关于 → 沉浸光感」。
   * · api — 设备 API 版本 (<26 一定没有材质)
   * · level — 设备算力档位 (EXQUISITE/GENTLE/SMOOTH), SMOOTH 只有底色没有滤色
   * · state — module.json5 里 ohos.arkui.UIMaterial.state 的解析结果
   * · filter — 是否支持沉浸式材质 (false 时所有材质都是空操作)
   */
  static diagnostics: string = '';

  /** 底部页签栏专用: 比普通卡片更"实"一点, 压在浅色地图上也要看得清字 */
  private static barMaterial: uiMaterial.ImmersiveMaterial | undefined = undefined;

  /**
   * 初始化材质。在 EntryAbility.onCreate 中调用一次即可。
   * 低版本设备 / 不支持材质的设备会静默跳过, 各页面拿到的都是 undefined。
   */
  static init(): void {
    // 已经成功就不再重复初始化; 失败则允许重试
    // (实测: 在 Ability.onCreate 阶段调用 isImmersiveMaterialSupported() 可能返回 false,
    // 等窗口/UI 上下文就绪后再调一次就正常了)
    if (MaterialKit.supported) {
      return;
    }
    try {
      if (deviceInfo.sdkApiVersion < 26) {
        MaterialKit.lastError = `api ${deviceInfo.sdkApiVersion}`;
        hilog.info(DOMAIN, TAG, 'immersive material skipped (api ${public}d)', deviceInfo.sdkApiVersion);
        return;
      }
      const deviceSupported: boolean = uiMaterial.isImmersiveMaterialSupported();
      const level: uiMaterial.MaterialLevel = uiMaterial.getGlobalMaterialLevel();
      const info: uiMaterial.MaterialInfo = uiMaterial.getMaterialInfo();
      MaterialKit.diagnostics =
        `api ${deviceInfo.sdkApiVersion} / 档位 ${level} / 应用开关 state=${info.state} / filter=${deviceSupported}`;
      hilog.info(DOMAIN, TAG, 'material probe: api=${public}d level=${public}d state=${public}d filter=${public}s',
        deviceInfo.sdkApiVersion, level, info.state, JSON.stringify(deviceSupported));
      if (!deviceSupported) {

```

```

MaterialKit.lastError = `api ${deviceInfo.sdkApiVersion} / device=false`;
hilog.info(DOMAIN, TAG, 'immersive material not supported by device');
return;
}
// 底部页签栏：官方《沉浸光感 · 场景规范》里底部悬浮用 THIN + 渐变颜色蒙层。
// 这里的染色比普通卡片重一档：页签栏经常压在浅色地图上，
// 太透会让金色/灰色图标失去对比度（实测 #1F 透明度的染色在浅色底图上发飘）。
MaterialKit.barMaterial = new uiMaterial.ImmersiveMaterial({
  style: uiMaterial.ImmersiveStyle.THIN,
  materialColor: Palette.glassBar,
  colorInvert: false,
  applyShadow: true,
  interactive: false,
// 手指按在页签上时，材质表面浮起一层金色光晕（沉浸光感的"光感"部分）
lightEffect: { color: MaterialKit.GLOW }
});
MaterialKit.supported = true;
MaterialKit.lastError = '';
hilog.info(DOMAIN, TAG, 'immersive material ready');
} catch (err) {
MaterialKit.lastError = `exception: ${JSON.stringify(err)}`;
hilog.warn(DOMAIN, TAG, 'immersive material unavailable: ${public}s', JSON.stringify(err));
}
}

/** 当前设备是否真的用上了沉浸光感材质 */
static isSupported(): boolean {
  return MaterialKit.supported;
}

/**
 * 底部页签栏（Tabs.barFloatingStyle 专用）。
 *
 * 为什么单独一个：官方《组件适配沉浸光感》规定，Tabs 的悬浮页签栏要同时满足
 * `barOverlap(true) + vertical(false) + barPosition(BarPosition.End)` 才会吃到材质。
 * 这条路径是系统原生绘制的，比自己拿 Button 拼一条导航（材质只在部分容器生效）可靠得多。
 * 返回值类型必须是 ImmersiveMaterial（barFloatingStyle.systemMaterial 的形参类型），
 * 不能用基类 Material，否则 ArkTS 编译期类型不匹配。
 */
static floatingTabBar(): uiMaterial.ImmersiveMaterial | undefined {
  return MaterialKit.barMaterial;
}
}

// ===== 文件: common/PrivacyConsent.ets =====
import { preferences } from '@kit.ArkData';
import { common } from '@kit.AbilityKit';
import { hilog } from '@kit.PerformanceAnalysisKit';

const DOMAIN: number = 0x0000;

```

```

const TAG: string = 'PrivacyConsent';
const STORE_NAME: string = 'privacy_store';
const KEY_AGREED_VERSION: string = 'agreed_version';

/**
 * 当前隐私政策版本。
 * 政策内容有实质修改时把版本号 +1，用户下次启动会被重新询问一次
 * （这是《个人信息保护法》与华为应用市场审核的通行做法）。
 */
export const PRIVACY_VERSION: string = '1.0';

/**
 * 隐私政策同意状态。
 *
 * 为什么单独一个文件：华为应用市场审核明确要求「App 首次运行时应当以显著方式
 * 提示用户阅读隐私政策，并征得用户同意后再收集个人信息」，所以同意状态必须
 * **先于任何数据采集**落盘可读。这里用轻量的 preferences 存，
 * 不依赖业务数据库（数据库初始化本身也要等 UI 起来）。
 */
export class PrivacyConsent {
  private static store: preferences.Preferences | undefined = undefined;
  private static agreed: boolean = false;

  /** 启动时调用一次，把上次的同意状态读回内存 */
  static async init(context: common.UIAbilityContext): Promise<void> {
    if (PrivacyConsent.store !== undefined) {
      return;
    }
    try {
      const options: preferences.Options = { name: STORE_NAME };
      const store: preferences.Preferences = await preferences.getPreferences(context, options);
      PrivacyConsent.store = store;
      const version: string = store.getSync(KEY_AGREED_VERSION, '') as string;
      PrivacyConsent.agreed = version === PRIVACY_VERSION;
      hilog.info(DOMAIN, TAG, 'consent restored: agreed=%{public}d version=%{public}s',
        PrivacyConsent.agreed ? 1 : 0, version);
    } catch (err) {
      // 读不到就按"未同意"处理：宁可多问一次，也不替用户做决定
      PrivacyConsent.agreed = false;
      hilog.warn(DOMAIN, TAG, 'consent init failed: %{public}s', JSON.stringify(err));
    }
  }

  static isAgreed(): boolean {
    return PrivacyConsent.agreed;
  }

  /** 用户点了"同意并继续" */
  static async agree(): Promise<void> {
    PrivacyConsent.agreed = true;
  }

```

```

const store: preferences.Preferences | undefined = PrivacyConsent.store;
if (store === undefined) {
  return;
}
await store.put(KEY_AGREED_VERSION, PRIVACY_VERSION);
await store.flush();
hilog.info(DOMAIN, TAG, 'consent granted');
}
}

// ===== 文件: common/Theme.ets =====
/**
 * 全局视觉规范（对应《设计前指导书》3.4 节）
 *
 * 设计语言：「暗夜金脉」—极夜蓝黑做底，金色代表"已点亮"。
 * 鸿蒙 7 适配：所有悬浮面板 / 卡片 / 控件都在 backgroundColor 之外
 * 叠加一层 uiMaterial 沉浸光感材质（见 common/MaterialKit.ets），
 * 低版本设备自动退回纯色，不会出现"半透明看不见"的问题。
 */
export class Palette {
  /** 页面底色：极夜蓝黑 */
  static readonly bg: string = '#070B14';
  /** 卡片底：比底色亮一档，作为材质的降级底色 */
  static readonly surface: string = '#111A29';
  static readonly surfaceSoft: string = '#0C1320';
  static readonly line: string = '#1E2A40';
  /** 主文字：纯白。沉浸光感材质上的文字必须是纯白才够醒目（灰色在玻璃上会发闷） */
  static readonly text: string = '#FFFFFF';
  static readonly textDim: string = '#8A99B5';
  static readonly gold: string = '#F5C542';
  static readonly goldLight: string = '#FFD97D';
  /** 深金：金色文字压在浅色底图上时用 */
  static readonly goldDeep: string = '#C8861A';
  static readonly cyan: string = '#22D3EE';
  static readonly danger: string = '#FF6B6B';
  static readonly unlitFill: string = '#22304A';
  static readonly unlitStroke: string = '#3B4A69';

  /** 材质染色：必须是带透明度的颜色，太实会挡掉沉浸光感的滤色 */
  static readonly glassPanel: string = '#2E0A1020';
  static readonly glassCard: string = '#1F0A1020';
  static readonly glassControl: string = '#24101828';
  /**
 * 底部悬浮页签栏染色：比卡片重一档。
 * 页签栏经常压在浅色地图上，染色太淡时金色/灰色图标对比度不够。
 */
  static readonly glassBar: string = '#5E0A1020';
  /** 金色光晕（用于"已点亮"的发光感） */
  static readonly goldGlowSoft: string = '#59F5C542';
  static readonly goldGlowFaint: string = '#1FF5C542';

```

```

}

/**
 * 地图标注视觉规范。
 * 目标是"看起来像底图自己画出来的": 无底板、白色描边、深灰字;
 * 已点亮城市换成琥珀金, 在浅色底图上仍有足够对比度, 同时保留"金色 = 去过"的语言。
 */
export class MapLabelStyle {
  /** 白色描边 (halo), 保证文字压在山脉 / 河流等复杂底纹上依然可读 */
  static readonly halo: string = '#FFFFFF';
  /**
   * 与底图标签完全一致的文字色 (近黑)。
   * 取样方式: 在底图国名"中华人民共和国"的字形核心 (笔画内部) 取最密集像素,
   * 实测 RGB(46~52) → 取 #303030。注意不要取抗锯齿边缘, 否则会得到偏灰的错值。
   */
  static readonly labelText: string = '#303030';
  /** 已点亮城市: 琥珀金 (比 goldLight 深, 避免在浅色底图上发飘) */
  static readonly visitedText: string = '#C8861A';
}

export class Dimens {
  static readonly pagePadding: number = 16;
  static readonly cardRadius: number = 20;
  static readonly pillRadius: number = 24;
  static readonly fabSize: number = 46;
  /** 悬浮底部导航高度 (含内边距) */
  static readonly navHeight: number = 62;
  /** 页面内容为悬浮导航预留的底部空间 */
  static readonly navSpace: number = 96;
  static readonly gap: number = 12;
}

/**
 * 动效规范。鸿蒙 7 的"沉浸光感"强调交互要有物理反馈:
 * 按下有形变、状态切换有过渡, 而不是生硬地跳变。
 */
export class Motion {
  /** 按压形变比例 */
  static readonly pressScale: number = 0.94;
  /** 常规过渡时长 (毫秒) */
  static readonly normal: number = 240;
  /**
   * 底部页签切换时长 (毫秒)。
   * 比 normal 稍长一点, 配合弹簧曲线收尾, 让"地图 → 我的"这种跨三页的跳转也有落定感。
   */
  static readonly pageSwitch: number = 320;
  /** 缓慢过渡 (用于光晕、进度) */
  static readonly slow: number = 420;
}

```

```

/** 地图容器初始相机位置：中国全图 */
export class MapDefaults {
/**
* 初始镜头：必须把中国国土完整框进来（西起 73°E 的帕米尔高原，东到 135°E 的黑龙江入海口）。
*
* 实测取舍：zoom 3.6 经度视野只有约 46°，新疆会被切掉；zoom 3.1 视野约 65°，
* 但标签相对地图太大、东部省份互相压字。取 **3.4（视野约 52°）**：
* 既能同时容下新疆（85°E）与黑龙江（127°E），又比 3.1 松快一档。
*/
static readonly centerLat: number = 35.5;
static readonly centerLng: number = 105.5;
static readonly zoom: number = 3.4;
/**
* 点按地图时，判定为“想点亮这座城市”的最大**屏幕**距离（vp）。
*
* 为什么不用公里：同一个屏幕距离，在不同缩放级别对应的公里数差几十倍。
* 全国视野下 60 公里只有约 12 物理像素（点位本身也只有约 12 像素宽），
* 手指落点根本压不进去——这就是“一个城市要点好几次才弹页面”的原因。
* 44vp ≈ 一根手指的落点半径，也是系统对最小可点区域的经验值。
*/
static readonly tapRadiusVp: number = 56;
/**
* 自己识别“点按”的位移容差（vp）。
* 手指点一下通常会抖动几像素，地图 SDK 会把它当成拖拽从而**不回调 mapClick**——
* 这就是“一个城市要点好几次”的另一半原因。我们在 MapComponent 上自己收 onTouch，
* 位移小于这个值、且时长够短，就判定为一次点击。
*/
static readonly tapSlopVp: number = 16;
/** 判定为点击的最长按压时间（毫秒），超过按长按处理 */
static readonly tapMaxMs: number = 500;
/**
* 已点亮城市点位的闪烁参数。
* 每座城市按行政区划代码派生出一个**不同的周期**（0.9~1.9 秒）和**不同的相位**，
* 于是 370 个点各闪各的、永不同步——一眼就能看出哪些地方去过。
*/
static readonly blinkMinPeriodMs: number = 900;
static readonly blinkPeriodRangeMs: number = 1000;
/** 闪烁时的透明度区间：暗到亮 */
static readonly blinkDimAlpha: number = 0.25;
static readonly blinkBrightAlpha: number = 1.0;
/** 刷新闻隔（毫秒）：80ms ≈ 12.5fps，兼顾顺滑与耗电 */
static readonly blinkIntervalMs: number = 80;
/**
* 分级标注阈值（按缩放级别三段式，互不重叠）：
* zoom < capitalZoom (4.5) → 只标省名（+ 底图自带的国家名）
* capitalZoom ≤ zoom < nameZoom → 只标 34 个省会 / 首府（此时不标省名）
* zoom ≥ nameZoom (7) → 铺开全部 370 个地级市名
*
* 阈值取 4.5 的原因：底图自带样式在 zoom≈4 就开始画省名（实测截图确认），
* 若我们的省名拖到 5 才退场，中间会出现“同一省份两个名字”的重复。

```

```

* 已在 Petal Maps Studio 里把「省份」设为隐藏，从源头消除重复。
*/
static readonly capitalZoom: number = 4.5;
/** 达到该缩放级别才显示城市名标注 */
static readonly nameZoom: number = 7;
/**
* 单屏最多显示多少个地级市名。
* 由于自绘标签之间已做碰撞避让（见 MapPage.labelConflicts），
* 这个上限只用于兜底性能，不再承担“防糊成一团”的职责。
*/
static readonly maxLabels: number = 120;
}

// ===== 文件: common/WindowService.ets =====
import { common } from '@kit.AbilityKit';
import { window } from '@kit.ArkUI';
import { hilog } from '@kit.PerformanceAnalysisKit';

const DOMAIN: number = 0x0000;
const TAG: string = 'WindowService';

/**
* 沉浸式窗口的“安全区”测量。
*
* 为什么需要：
* App 开启了 setWindowLayoutFullScreen(true)，内容会铺到状态栏 / 手势条底下。
* 地图、底图这类“背景层”希望铺满（沉浸感），
* 但顶部信息卡、悬浮按钮必须避开状态栏与手势条，否则会像“压住时间电量”那样难看、也点不到。
*
* 用法：EntryAbility 启动后调用 WindowService.measure(),
* 页面在 aboutToAppear 里读取 WindowService.safeTop / safeBottom（单位 vp）。
*/
export class WindowService {
  /** 状态栏高度（vp），顶部内容需要避开这么多 */
  static safeTop: number = 0;
  /** 底部手势条 / 导航栏高度（vp） */
  static safeBottom: number = 0;

  static async measure(context: common.UIAbilityContext): Promise<void> {
    try {
      const mainWindow: window.Window = await window.getLastWindow(context);
      WindowService.applyAvoidAreas(mainWindow);
      // 按官方《开发应用沉浸式效果》的建议：注册监听，动态获取避让区变化
      // （窗口在全屏/悬浮/分屏间切换、旋转、折叠屏展开折叠时都会回调）
      mainWindow.on('avoidAreaChange', () => {
        WindowService.applyAvoidAreas(mainWindow);
      });
      hilog.info(DOMAIN, TAG, 'safe area top=%{public}d bottom=%{public}d',
        WindowService.safeTop, WindowService.safeBottom);
    } catch (err) {

```

```
// 取不到就退回一组保守值, 保证内容不会被状态栏压住
WindowService.safeTop = 38;
WindowService.safeBottom = 16;
hilog.warn(DOMAIN, TAG, 'measure failed, use fallback: %{public}s', JSON.stringify(err));
}
}

/**
 * 读取避让区。
 * 顶部用 TYPE_SYSTEM (状态栏) ;
 * 底部**必须用 TYPE_NAVIGATION_INDICATOR** (导航条/手势条) ,
 * 这是官方文档明确给出的取法—TYPE_SYSTEM 的 bottomRect 在手势导航机型上通常是 0,
 * 用它算底部间距会让可交互元素贴到手势条上。
 */
private static applyAvoidAreas(mainWindow: window.Window): void {
  try {
    const systemArea: window.AvoidArea =
      mainWindow.getWindowAvoidArea(window.AvoidAreaType.TYPE_SYSTEM);
    WindowService.safeTop = px2vp(systemArea.topRect.height);
  } catch (err) {
    WindowService.safeTop = 38;
  }
  try {
    const navArea: window.AvoidArea =
      mainWindow.getWindowAvoidArea(window.AvoidAreaType.TYPE_NAVIGATION_INDICATOR);
    WindowService.safeBottom = px2vp(navArea.bottomRect.height);
  } catch (err) {
    WindowService.safeBottom = 16;
  }
}

// ===== 文件: entrybackupability/EntryBackupAbility.ets =====
import { hilog } from '@kit.PerformanceAnalysisKit';
import { BackupExtensionAbility, BundleVersion } from '@kit.CoreFileKit';

const DOMAIN: number = 0x0000;

export default class EntryBackupAbility extends BackupExtensionAbility {
  async onBackup(): Promise<void> {
    hilog.info(DOMAIN, 'TravelChinaFootprint', 'onBackup ok');
    await Promise.resolve();
  }

  async onRestore(bundleVersion: BundleVersion): Promise<void> {
    hilog.info(DOMAIN, 'TravelChinaFootprint', 'onRestore ok %{public}s', JSON.stringify(bundleVersion));
    await Promise.resolve();
  }
}
```